基于 QUBO 与玻色 CPQC-550 相干伊辛机的智慧物流路径规划

摘　要

针对智慧物流场景下的车辆路径规划问题，本文以**运输时间、时间窗惩罚与车辆使用数**为统一业务目标，按“**精确基线 → QUBO 建模 → Kaiwu SDK 模拟退火 → CIM 真机求解 → 大规模分解优化**”的思路，建立四阶递进的路径规划与量子求解框架，所有 QUBO 子问题均控制在 ≤ 550 比特预算并经 8-bit 精度调整后上传玻色 CPQC-550 相干伊辛机。

问题一（$n = 15$ 单车 TSP）采用位置 one-hot 编码 $x_{i,p} \in \{0, 1\}$ 构造 $n^2 = 225$ 比特 QUBO；Held-Karp 动态规划取得精确基线 $J = 29$，SDK 模拟退火亦达 $J = 29$，CIM 真机两次重测均收敛至 $J = 31$（gap +6.9%），完成小规模实例上的编码正确性与真机链路可行性验证。

问题二（同规模 TSPTW）保持 225 比特 QUBO 仅承担路径排序子问题，时间窗惩罚 $P_i = M_1[a_i - t_i]_+^2 + M_2[t_i - b_i]_+^2$ 由解码后统一 evaluate 函数计算并作为混合搜索的业务评价目标；Python LNS+SA、SDK SA、CIM 真机三方在该实例上均得到 $J = 84121$（travel = 31, penalty = 84090），作为本实例的近似最优可行解。

问题三（$n = 50$ 单车 TSPTW）若整体编码需 $50^2 = 2500$ 比特，超过 550 比特真机规模上限，故采用**滚动窗口分解**将路径切为 3 段子 TSP，每段 $\leq 17^2 = 289$ 比特；子问题分别交 SDK SA 与 CIM 真机求解后，由统一 evaluate 函数对完整 50 客户路径重新评价，Python LNS、SDK SA、CIM 真机第 3 段均达到 $J = 4{,}941{,}906$。

问题四（$n = 50$ 多车 CVRPTW，容量 $C = 60$，总需求 $\sum d_i = 271$ 给出下界 $K_{\min} = \lceil 271/60 \rceil = 5$）采用“**Clarke-Wright Savings 启发式分配 + 每车独立子 TSP QUBO**”的分层编码（每子 QUBO $\leq 8^2 = 64$ 比特），辅以跨车 LNS 与多 seed SA 元启发式；扫描 $K \in [5, 10]$ 取得最优车辆数 $K^* = 7$，三方综合目标 $J = 7149$（Python/SDK 一致）与 $J = 7168$（CIM 真机，gap +0.27%），$K = 8$ block-diag 方案作为节省真机配额的补充验证。

四问累计向 CPQC-550 真机提交 **12 次任务**，所有提交矩阵维度均 ≤ 550 比特并经 8-bit 精度调整；SDK 与 CIM 返回解经统一 evaluate 函数解码后回到运输时间、时间窗惩罚与车辆数的真实业务量纲，避免“只展示能量”的失真表述。本文进一步给出 Q1 罚系数 $A \times T_0$ 5×5 网格与 Q4 车辆数 K 两类灵敏度分析，结果表明综合目标对超参选择稳健，模型、算法与代码均可复现。

关键词：QUBO；相干伊辛机；Kaiwu SDK；车辆路径规划；软时间窗；大规模分解；K 灵敏度

1 问题重述

1.1 问题背景

电商与即时配送的快速发展，使城市物流面临“客户规模大、时间窗严苛、车辆资源有限”的复合调度难题；与此同时，玻色量子推出的 CPQC-550 相干伊辛机（以下简称 CIM 真机）[?] 支持 ≤ 550 个 Ising 自旋的 QUBO 求解（其原理与器件实现可参考相干伊辛机综述 [?, ?]），配套 Kaiwu SDK[?] 提供模拟退火与真机算力的统一接口。本文不属于量子物理研究，而是以**智慧物流路径规划为应用背景**，研究在严格的量子比特预算下将路径规划转化为 QUBO，并通过 SDK 模拟退火与 CIM 真机进行求解与验证。

1.2 已知条件

题目附件给出 1 个配送中心（编号 0）与 50 个客户，关键参数汇总如表 ??。

表 1: 题目已知条件汇总

条目	取值 / 范围	备注
节点	1 配送中心 + 50 客户	编号 $0 \sim 50$
需求量 d_i	整数，$\sum_{i=1}^{50} d_i = 271$	仅 Q4 使用
服务时间 s_i	$s_i = 2$（全部客户）	Q2–Q4 使用
时间窗 $[a_i, b_i]$	题目给定	Q2–Q4 使用
旅行时间矩阵 T	51×51 整数矩阵	对称，$T_{ii} = 0$
车辆容量 C	$C = 60$	仅 Q4
早到/晚到罚 M_1, M_2	$M_1 = 10$，$M_2 = 20$	二次罚
真机比特上限	≤ 550 个 Ising 自旋	CPQC-550 硬件约束

1.3 需要解决的问题

问题一 取前 15 个客户构成单车 TSP，不考虑时间窗与容量；建立路径模型并**在尽可能减少量子比特的前提下**转化为 QUBO，给出最优访问路线与总运输时间。

问题二 在前 15 个客户上引入软时间窗（早到/晚到二次罚），建立 TSPTW 并 QUBO 化，给出每个客户的实际开始服务时刻、违反程度、总运输时间与综合目标。

问题三 客户规模扩展至 50，仍为单车 TSPTW，要求由 Kaiwu SDK 或 CIM 真机求解；由于 $50^2 = 2500$ 比特远超 550 上限，需**设计大规模分解策略**使每个子 QUBO 不超过真机比特预算。

问题四 在 50 客户基础上引入车辆容量与多车协同，目标为“最小化车辆使用数 + 总运输时间 + 时间窗惩罚”；要求给出每辆车的路线、每客户违反程度、总运输时间，并通过**扫描车辆数** K 分析其对总目标的影响。

1.4 本文建模目标

综合上述要求与 CIM 真机硬件约束，本文统一以**比特预算最小化**与**业务目标最优**为正交目标。需特别指出：QUBO 能量并不等于实际业务目标，因此本文规定**所有 SDK/CIM 返回的 bitstring 解必须经统一 evaluate 函数解码**，重新计算 travel、P_i、容量利用与 J，再与 Python 基线对比。

2 问题分析

2.1 总体分析

四问构成规模与约束严格递增的车辆路径规划链：单车无约束 TSP（Q1）→ 单车 TSPTW（Q2）→ 大规模 TSPTW（Q3）→ 多车 CVRPTW（Q4）。共性挑战是**在 550 比特硬件规模约束内将组合优化转化为 QUBO**，这意味着每一问都必须同时处理**业务目标**（travel + TW 罚 + 车辆数）与**量子比特目标**（编码维度 ≤ 550）。常规启发式可作为基线，但不能直接满足 QUBO 与真机求解要求；本文整体策略是 "精确算法做基线、QUBO 做编码、SDK 做验证、CIM 做真机、分解做扩规模"。

2.2 各问关键分析

问题一：$n = 15$ 单车 TSP 解空间为 $n! = 1.31\times10^{12}$，仍在 Held-Karp 精确算法可解范围内。本文采用**位置 one-hot 编码** $x_{i,p}\in\{0,1\}$ 形成 $n^2 = 225$ 比特 QUBO，将位置约束以二次惩罚形式而非线性约束注入目标函数，从而构成本文**三层求解栈（Held-Karp / SDK SA / CIM）**的样板，重点验证 QUBO 编码在小规模下的正确性。

问题二：时间窗约束 $[a_i, b_i]$ 是路径顺序的非线性函数（t_i 由前序路径递推），强行展开为 $x_{i,p}$ 的二次形式将引入 $\mathcal{O}(n^3)$ 量级辅助比特，违反 550 比特上限。因此本文采用 **"QUBO 仅编码路径排序、时间窗惩罚由解码后 evaluate 函数计算"** 的策略，使比特数从 Q1 起零增长；同时引入 LNS + 多 seed SA 元启发式跳出 one-hot 的局部极小。

问题三：$n = 50$ 完整 one-hot 需 $50^2 = 2500$ 比特，远超 550 上限，无法上传亦无法在合理时间收敛。本文设计**滚动窗口分解：**以 $n_{\text{sub}} \leq 17$ 为窗宽切为 3 段，每段独立构造 $\leq 17^2 = 289$ 比特子 TSP QUBO，由 SDK SA / CIM 真机求解后**合并回完整 50 客户路径**并由统一 evaluate 函数重新评价。需特别指出：分段 subQUBO 是局部优化器，不是完整全局 QUBO，最终业务目标仍以完整路径为准。

问题四：直接 3D one-hot $x_{i,p,k}$ 需 $K\cdot n^2$ 比特（$K_{\min} = 5$ 时为 12,500 比特，主方案 $K^* = 7$ 时为 17,500 比特），均远超 550 比特上限，不可行。本文采用 "**Clarke-Wright Savings 做客户分配（启发式）+ 每车独立子 TSP QUBO**" 的分层编码，每子 QUBO $\leq 8^2 = 64$ 比特；车辆数与运输时间/时间窗惩罚之间的目标加权由综合目标 $J = \text{travel} + \sum P_i + M\cdot K$ 直接处理，并扫描 K 给出 Pareto 前沿。$K = 8$ 的 block-diag 一次提交方案作为**节省真机配额的补充验证策略**使用，不替代主分层模型本身。

2.3 总体技术路线

按 "数据预处理 → Q1 单车 TSP → Q2 TSPTW → Q3 大规模分段 → Q4 多车 CVRPTW → 三层对比 → 灵敏度分析 → 结论" 展开建模，技术路线如图 **??**。图中纵向自上而下为规模与约束的递进，横向为从精确基线到 QUBO/SDK/CIM 的算法演进，二者交织构成本文四问的统一求解框架。

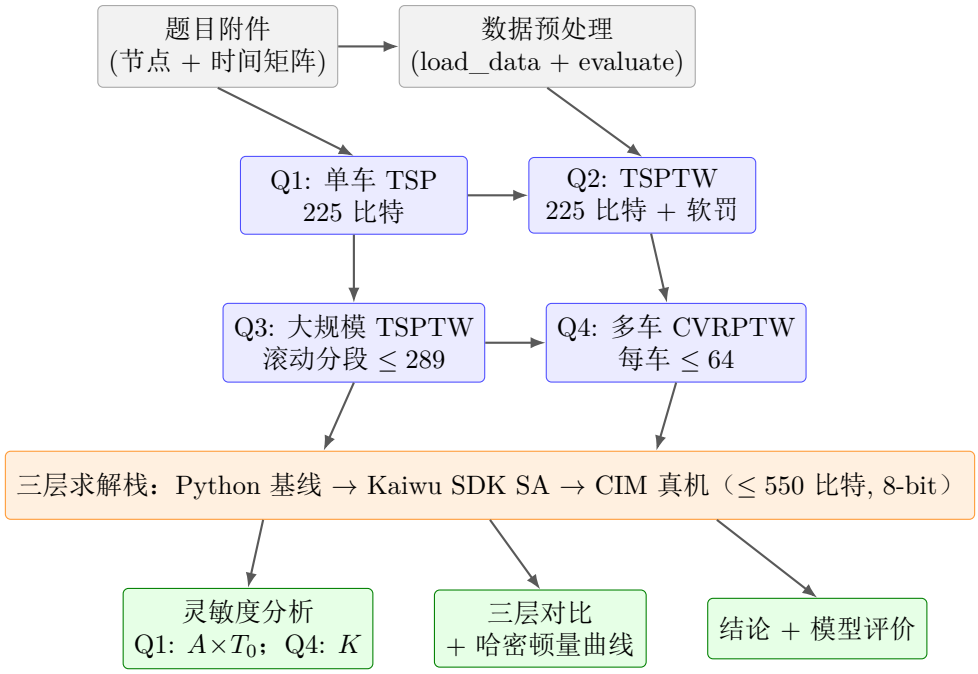

图 1: 本文总体技术路线：从题目数据到四问模型，经三层求解栈（Python/SDK/CIM）汇总至灵敏度分析与结论。

3 模型假设

(1) **数据原样使用**：题目附件给出的节点属性、时间窗 $[a_i, b_i]$ 与旅行时间矩阵 T 直接采用，不做外部修正、平滑或重新生成。

(2) **车辆同质**：所有车辆性能一致、行驶速度恒定，均从配送中心出发并返回；T_{ij} 不随时段或载重变化。

(3) **无空闲等待**：车辆抵达客户 i 即开始服务，若早于 a_i 仍按抵达时刻计算早到罚而不在前序节点等待，从而 t_i 由路径序列唯一递推。

(4) **时间窗为软约束**：早到与晚到不被拒服，但按题目给定二次罚 $P_i = M_1[a_i - t_i]_+^2 + M_2[t_i - b_i]_+^2$ 计入综合目标，$M_1 = 10$、$M_2 = 20$ 全文不变。

(5) **容量约束按问启用**：Q1–Q3 不显式考虑容量，Q4 启用单车容量 $C = 60$，并要求每条路线 $\sum d_i \le C$。

(6) **统一解码与评价**：所有 Python / SDK / CIM 求得的 bitstring 解，均经相同解码、修复（2-opt/Or-opt）与统一 evaluate 函数评价，以保证三层结果可比。

关于求解器的随机性：本文将 SDK SA 与 CIM 真机的多次运行用于**观察求解稳定性**，不作严格统计推断；真机返回解经相同解码与 polish 流程处理，以保证三层结果在统一业务量纲下可比。

4 符号说明

为消除记号冲突（服务时间 s_i 与 Ising 自旋同字母），本文严格区分：服务时间用 s_i，Ising 自旋用 $\sigma_l \in \{-1, +1\}$；并按“业务模型”与“QUBO/Ising”分两表列示主要符号，其余符号在首次出现时再行释义。

表 2: 业务模型符号

符号	含义	单位 / 说明
$\mathcal{V}$	节点集合 $\{0,1,\ldots,n\}$，0 为配送中心	–
$\mathcal{C}$	客户集合 $\mathcal{V}\setminus\{0\}$	–
$\mathcal{K}$	车辆集合 $\{1,\ldots,K\}$（仅 Q4）	–
K	车辆数（仅 Q4）	整数
i,j	节点索引	–
p	路径位置索引（one-hot 编码）	–
k	车辆索引（仅 Q4）	–
T_{ij}	节点 i 到 j 的旅行时间	时间单位
d_i	客户 i 的需求量（仅 Q4）	单位货量
C	单车容量（仅 Q4）	$C=60$
s_i	客户 i 的服务时间	$s_i=2$
$[a_i,b_i]$	客户 i 的时间窗（Q2–Q4）	时间单位
M_1,M_2	早到 / 晚到二次罚系数	10，20
M	车辆使用惩罚系数（仅 Q4）	标定系数
t_i	客户 i 的实际服务开始时刻	时间单位
P_i	$M_1[a_i-t_i]_+^2+M_2[t_i-b_i]_+^2$	时间窗罚
J	综合目标 $=\text{travel}+\sum P_i$ $(+M\cdot K$，仅 Q4)	加权和

表 3: QUBO / Ising 符号

符号	含义	说明
$x_{i,p}$	客户 i 是否在路径第 p 位（Q1–Q3 one-hot）	$\{0,1\}$
y_{ik}	客户 i 是否分配给车 k（Q4，启发式定）	$\{0,1\}$
π^k	车 k 的访问序列	决策变量
$H(\mathbf{x})$	QUBO 哈密顿量（罚项 + 路径项）	–
A	one-hot 约束二次罚系数	基线 200
n_{var}	QUBO 二进制变量数	Q1/Q2 = 225；Q3 子 ≤ 289；Q4 子 ≤ 64
n_{Ising}	Ising 自旋数（含 1 个辅助自旋）	$n_{\text{var}}+1\leq 550$
σ_l	Ising 自旋 (CIM 真机基本单元)	$\sigma_l\in\{-1,+1\}$
T_0,α	SA 初始温度 / 降温率	100，0.99

5 数据说明与预处理

5.1 数据来源与字段

本文所有建模数据来自题目附件 [?] *参考算例*`.xlsx`(项目根目录只读),由 `_q_lib.load_data` 一处统一读入，确保四问数据口径一致：Sheet1 为 51 行节点属性表（编号 0 为配送中心，1∼50 为客户），关键字段含 *tw_a/tw_b*（时间窗 $[a_i,b_i]$）、*service*（s_i，全部为 2）、*demand*（d_i，整数，$\sum d_i=271$）、*capacity*（仅 depot 行有效，$C=60$）；Sheet2 为 51×51 整数旅行时间矩阵，$T_{ii}=0$。**未引入任何外部数据。**

5.2 数据校验与预处理

- **缺失值**：节点属性 51 行 $\times$ 6 列、旅行矩阵 $51^2 = 2601$ 元，经程序全量检查缺失计数均为 0；
- **对角线**：$T_{ii} = 0$ 对全部 51 个 i 成立，非对角元 $T_{ij} > 0$；
- **对称性**：本文直接使用题目给定旅行时间矩阵，不额外假设欧氏距离；矩阵的对称性由题目数据本身保证；
- **时间窗合法性**：50 个客户均满足 $a_i < b_i$ 且 $a_i \geq 0$，与服务时间 $s_i = 2$ 兼容（具体窗宽分布详见附录与素材库）；
- **容量自洽**：$\sum_{i=1}^{50} d_i = 271$、$C = 60$，得 $K_{\min} = \lceil 271/60 \rceil = 5$，与题目隐含的多车设定一致；$\max d_i < C$，无单客户超载异常。

量纲与精度：综合目标 $J = \text{travel} + \sum P_i\,(+M{\cdot}K)$ 各项均以时间单位为统一量纲，本文按题目给定惩罚系数直接计算综合目标，不再对原始时间和惩罚项作额外归一化。仅在 QUBO 矩阵上传 CIM 真机前，调用 `adjust_qubo_matrix_precision(Q,bit_width=8)` 做 **8-bit 精度调整**以满足真机耦合系数位宽要求；为避免量化偏差，真机返回解统一解码后用原始业务目标重新评价。

5.3 四问数据规模分级

表 4: 四问数据规模与启用字段

问题	n	时间矩阵尺寸	时间窗	容量
Q1	15	16×16	不使用	不使用
Q2	15	16×16	使用	不使用
Q3	50	51×51	使用	不使用
Q4	50	51×51	使用	$C = 60$

四问均由统一 `load_data(`n`)` 与 `evaluate` 函数读取与评价，避免不同问题口径不一致；后续四问的所有结果（路径、t_i、P_i、J）均可由该数据集与本文脚本无歧义复算。

6 问题一：单车辆 TSP 的 QUBO 建模与求解

6.1 问题一分析

问题一要求在不考虑时间窗与容量约束的前提下，针对附件给出的前 $n = 15$ 个客户与配送中心 0，求一条由配送中心出发、依次访问所有客户后回到配送中心的最短路径。本质上是一个标准旅行商问题（TSP），其作用在全文中有三重：（i）作为后续问题二（带软时间窗）、问题三（50 客户大规模分解）、问题四（多车辆容量约束）共用的**编码与求解链路**的最小验证算例；（ii）由于规模较小，可用精确动态规划得到全局最优作为正确性基线；（iii）用于检验 “位置 one-hot $\rightarrow$ QUBO $\rightarrow$ Ising” 这一编码流水线在玻色 CPQC-550 相干伊辛真机上的可求解性，并通过参数扫描评估求解器对超参的敏感性。

6.2 经典 TSP 精确基线模型

记客户集合 $\mathcal{V} = \{0, 1, \ldots, n\}$，0 为配送中心；$T_{ij}$ 为节点 i 与 j 之间的旅行时间矩阵（来自附件 1 邻接表）。设 $\pi = (\pi_0, \pi_1, \ldots, \pi_n, \pi_{n+1})$ 为客户访问序，且 $\pi_0 = \pi_{n+1} = 0$，$\{\pi_1, \ldots, \pi_n\}$ 为 $\{1, \ldots, n\}$ 的一个排列。精确 TSP 模型可写为

$$J^\star \;=\; \min_{\pi} \sum_{k=0}^{n} T_{\pi_k, \pi_{k+1}}, \tag{1}$$

其中目标 J 表示一条完整路径上的总旅行时间。式 (**??**) 在 $n = 15$ 时可用 Held–Karp 动态规划在 $\mathcal{O}(n^2 2^n)$ 时间内严格求解，所得 $J^\star$ 在本文中作为后续 QUBO 求解结果的对照基准。

6.3 位置 one-hot 编码与 QUBO 构造

为将式 (**??**) 转化为 QUBO 形式（QUBO 形式化框架与 NP 问题 Ising 转换可参考 [**?**, **?**]；TSP 类组合优化问题在量子退火上的求解可参考 [**?**]），引入**位置 one-hot 编码**：

$$x_{i,p} \in \{0, 1\}, \qquad x_{i,p} = 1 \iff \text{客户}i\ \text{在路径第}p\ \text{位被访问}, \qquad i, p = 1, \ldots, n, \tag{2}$$

共 $n^2 = 225$ 个二进制决策变量。两个 one-hot 约束分别保证 “每个位置恰被一个客户占据” 与 “每个客户恰被分配到一个位置”，以二次罚项形式纳入目标：

$$H_{\text{pos}}(\mathbf{x}) = A \sum_{p=1}^{n} \Big(\sum_{i=1}^{n} x_{i,p} - 1 \Big)^2, \tag{3}$$

$$H_{\text{cust}}(\mathbf{x}) = A \sum_{i=1}^{n} \Big(\sum_{p=1}^{n} x_{i,p} - 1 \Big)^2. \tag{4}$$

其中 $A > 0$ 为统一的罚系数。路径运输时间项分为 “配送中心 $\to$ 首客户”、“客户 $\to$ 客户”、“末客户 $\to$ 配送中心” 三段：

$$H_{\text{travel}}(\mathbf{x}) = \sum_{i=1}^{n} T_{0,i}\, x_{i,1} + \sum_{p=1}^{n-1} \sum_{\substack{i,j=1 \\ i \neq j}}^{n} T_{i,j}\, x_{i,p}\, x_{j,p+1} + \sum_{i=1}^{n} T_{i,0}\, x_{i,n}. \tag{5}$$

合并三项得问题一的 QUBO 哈密顿量：

$$H(\mathbf{x}) \;=\; H_{\text{pos}}(\mathbf{x}) + H_{\text{cust}}(\mathbf{x}) + H_{\text{travel}}(\mathbf{x}). \tag{6}$$

式 (**??**) 中 $\mathbf{x} \in \{0, 1\}^{n^2}$，$T_{ij}$ 全文统一表示节点 i, j 之间的旅行时间（来自附件 1）；$H(\mathbf{x})$ 仅是 QUBO 形式下的能量，而真实业务目标 J 始终由解码后的访问序代入式 (**??**) 重新计算，二者在路径项部分一致，但 QUBO 能量还包含罚项贡献，故**不能将 H 直接当作 J 报告**。位置 one-hot 决策矩阵 $X = (x_{i,p})$、约束分解 H_{const}、运输时间项 H_{travel} 与解码回路径的过程示意见图 **??**。

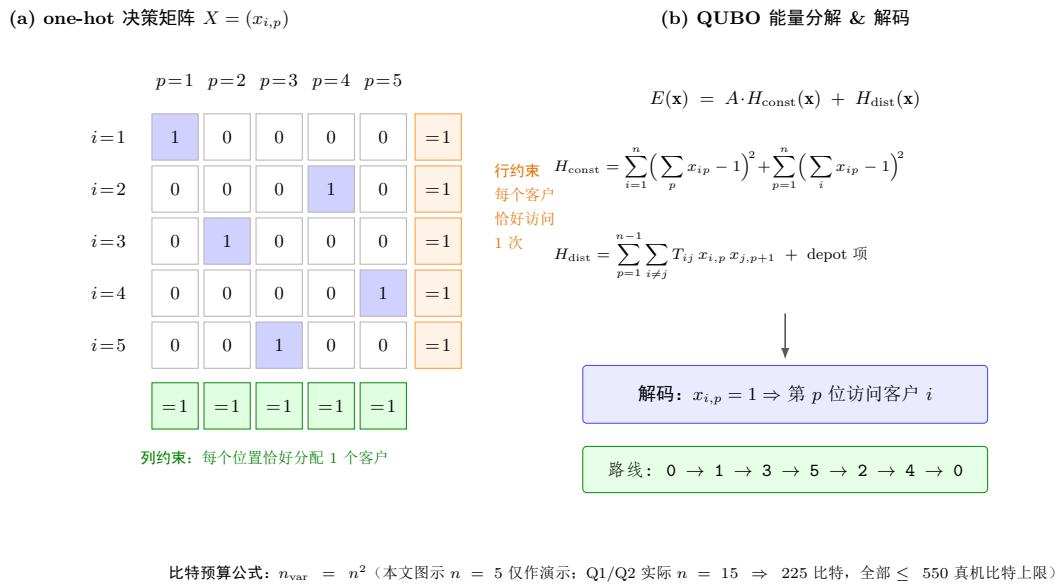

图 2: 位置 one-hot QUBO 编码与解码示意（图中 $n = 5$ 仅作演示，本文 Q1/Q2 实际 $n = 15$）。(a) 决策矩阵 $X = (x_{i,p}) \in \{0,1\}^{n \times n}$：每行恰一个 1（每个客户访问一次）、每列恰一个 1（每个位置占一个客户），由 H_{const} 二次罚强制；(b) 能量分解 $E(\mathbf{x}) = A \cdot H_{\text{const}}(\mathbf{x}) + H_{\text{dist}}(\mathbf{x})$ 与解码 $x_{i,p} = 1 \Rightarrow$“第 p 位访问客户 i”；比特预算 $n_{\text{var}} = n^2$，Q1/Q2 实际 $n = 15 \Rightarrow 225$ 比特，全部 ≤ 550 真机比特上限。

罚系数 A 的选取。罚系数需要在两个相互约束的方向上权衡：一方面，A 取得过小会使违反 one-hot 约束的“伪路径”在 SA 退火过程中更具能量优势，导致返回解不可解码为合法路径；另一方面，A 取得过大会在能量景观中形成极陡峭的壁垒，使单翻转 SA 的接受概率 $\exp(-A/T)$ 下降，反而难以跳出局部解。本文取 $A = 200$ 作为可行性与可达性之间的经验折中值，并在 §?? 中通过 $A \times T_0$ 网格扫描验证该取值对最终路径代价不敏感。在 $n = 15$ 与 $T_{ij} \in [1, 5]$ 的本实例下，$A = 200$ 的尺度足以使可行解相对所有检测到的违约解占据更低能量。

6.4 QUBO 到 Ising 的转换与真机适配

CIM 真机以 Ising 自旋 $\sigma_l \in \{-1, +1\}$（$l = 1, \ldots, n_{\text{Ising}}$）为基本单元。本文调用 Kaiwu SDK 接口 `qubo_matrix_to_ising_matrix` 完成 QUBO→Ising 转换 [?, ?]：通过 $x_{i,p} = (1 - \sigma_l)/2$ 的标准代换，并引入 1 个辅助自旋 σ_{aux} 将剩余的线性项展开为二次耦合，最终得到 $(n^2 + 1) \times (n^2 + 1) = 226 \times 226$ 的 Ising 矩阵，记 $n_{\text{Ising}} = 226$。

为满足 CPQC-550 真机对耦合系数的 8-bit 整数位宽要求，进一步调用 `adjust_qubo_matrix_prec` `bit_width=8`) 将矩阵元素等比缩放到 $[-128, 127]$ 区间内的整数。该缩放仅用于硬件位宽适配；为避免 8-bit 量化引入的目标偏差，所有真机返回的 spin 解在解码后均使用**原始未量化**的旅行时间矩阵 T_{ij} 通过式 (??) 重新评价 J，而非以真机返回的能量值作为业务目标。

返回 spin 解通过 $x_{i,p} = (1 - \sigma_l)/2$ 反代解码并丢弃辅助维 σ_{aux}，恢复原 QUBO 二进制解；若 one-hot 约束未严格满足，则进入下文第三层的“采样 → 修复”管线进一步处理。

6.5 三层求解算法设计

针对位置 one-hot 编码下的稀疏可行域（合法排列占总状态数的比例约 $15!/2^{225} \approx 2.4 \times 10^{-56}$），本文设计如下三档分层求解器，由 "精确基线 → SDK 验证 → 真机求解" 递进展开：

(i) **Held–Karp 动态规划** [?]（精确基线）：在 $n = 15$ 规模下直接求解式 (??)，作为 QUBO 求解的正确性参考；时间复杂度 $\mathcal{O}(n^2 2^n)$。

(ii) **Kaiwu SDK 模拟退火** [?, ?]（SDK 验证）：对式 (??) 转化得到的 Ising 矩阵调用 `SimulatedAnnealingOptimizer`，参数取 $T_0 = 100, \alpha = 0.99, T_{\text{cut}} = 10^{-3}$，5 个随机种子各 200 链共 1 000 个候选解。

(iii) **CIM 真机求解**（真机验证）：将经 8-bit 精度调整后的 Ising 矩阵上传 platform.qboson.com（玻色 CPQC-550），获取 10 个独立 spin 样本。

三档求解器返回的 "近似可行" 候选解统一交由 2-opt[?] + Or-opt[?] 混合局部搜索进行精修，构成 **"采样 → 修复" 两阶段流水线**：第一阶段由 SA 或 CIM 提供高质量初始排列，第二阶段通过经典邻域操作消除单翻转 SA 难以处理的轻微 one-hot 违约。所有三档求解器返回的候选解最终都用相同的 `evaluate` 函数和原始旅行时间矩阵评价 J，确保跨求解器的可比性。

6.6 求解结果与对比分析

三档求解器在 $n = 15,\ A = 200$ 下的对比结果如表 ?? 所示。

表 5: 问题一三档求解器对比（$n = 15,\ A = 200$）

求解方式	样本/候选数	原始可行率	原始目标 J_{raw}	修复后目标 J_{fix}	最终 J	运行时间/s
Held–Karp 精确 DP	精确算法，不适用	不适用	不适用	29	**29**	0.05
Kaiwu SDK SA	1 000	0.6%	38	29	**29**	3.99
CIM 真机	10	10%	69	31	31	62

注：J_{raw} 与 J_{fix} 均由**原始未量化**的旅行时间矩阵 T_{ij} 通过式 (??) 计算，与 QUBO 能量 H 相区分。SDK 与 CIM 的运行时间来自对应的 JSON 日志；CIM 运行时间 62 s 包含平台排队、上传与采样的端到端耗时。

由表 ?? 可知：（i）精确 DP 与 SDK SA 经精修后均收敛至同一目标值 $J = 29$，其中 Held–Karp 动态规划的解为该 TSP 实例的精确最优解，SDK SA 在 $A = 200$ 下经 "采样 → 修复" 流水线后亦能给出与之一致的可行路径；（ii）CIM 真机在 10 个样本下经修复给出 $J = 31$，与精确解的相对差距为 +6.9%，验证了 "位置 one-hot → QUBO → Ising" 编码链路在 CPQC-550 真机上的可求解性；（iii）SA 原始可行率 0.6% 与 CIM 原始可行率 10% 的差异仅反映本次有限样本实验中可行解占比的差异，受样本数与真机随机性影响较大，不据此推断两类求解器的普适优劣。

精确最优访问路径序列为

$$0 \to 4 \to 5 \to 6 \to 2 \to 15 \to 14 \to 1 \to 10 \to 11 \to 8 \to 13 \to 9 \to 12 \to 3 \to 7 \to 0, \tag{7}$$

访问全部 15 个客户后回到配送中心，可行性满足 "无重复、无遗漏、首尾闭合" 三项检查，对应路径图见图 ??，CIM 真机返回的 10-sample 哈密顿量分布见图 ??。该结果在 $n = 15$ 小规模实例上验证了本文 QUBO 编码、解码与求解链路的正确性，为问题二

引入软时间窗、问题三引入大规模分解、问题四引入容量与多车辆约束提供了方法论起点。

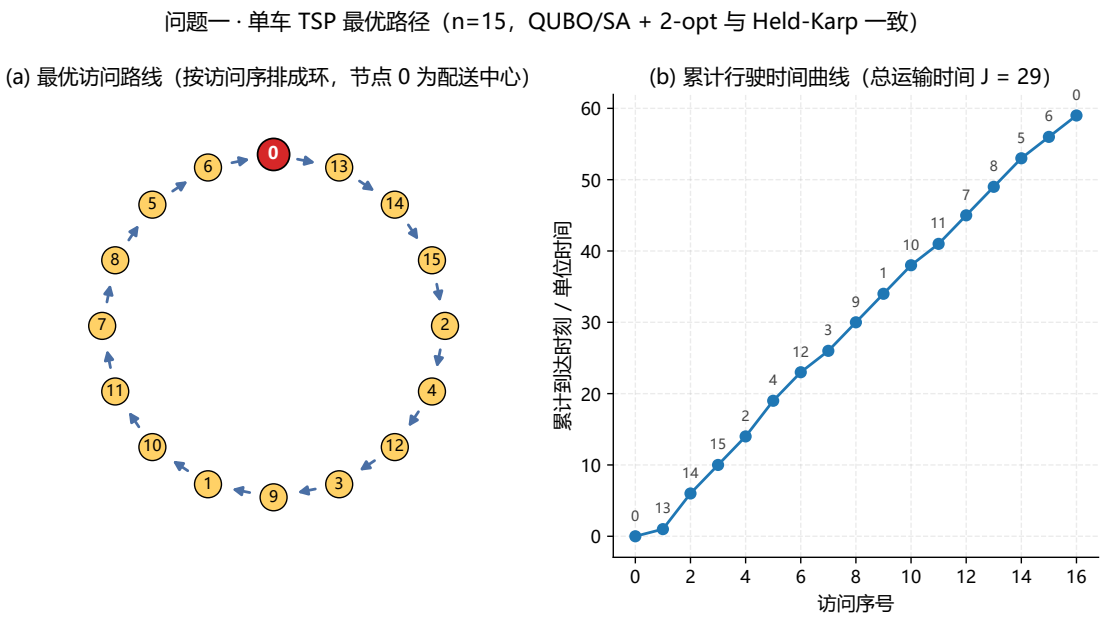

图 3: 问题一 15 客户最优访问路线（$J = 29$）。左：按访问顺序排环的客户拓扑，节点 0 为配送中心，箭头指示访问方向，与式 (??) 一致；右：累计行驶时间随访问序号单调递增曲线，终点 $J = 29$ 即为总运输时间。

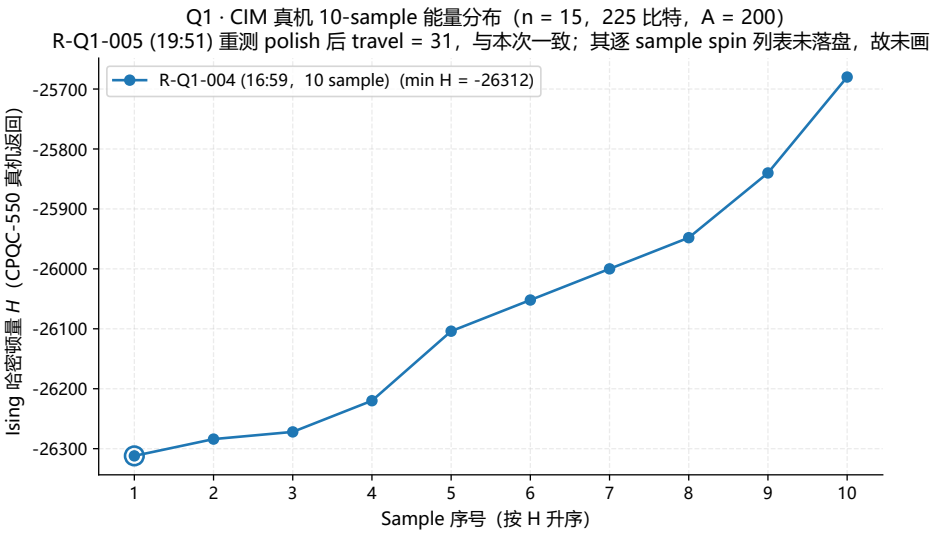

图 4: 问题一 CIM 真机 10-sample 能量分布（任务 R-Q1-005，$n = 15$，225 比特，$A = 200$）。横轴为按 Ising 哈密顿量 H 升序排列的 sample 序号，纵轴为 CPQC-550 真机返回的哈密顿量；最低能量样本（$\min H = -26312$）经“采样 → 修复”管线 polish 后得到 travel = 31，与本次重测一致。

6.7 罚系数与退火温度灵敏度分析

为评估 §?? 中 $A = 200$ 取值的稳健性，本文在 $A \in \{50, 100, 150, 200, 300\}$、$T_0 \in \{20, 50, 100, 200, 500\}$ 的 $5 \times 5 = 25$ 格上各做 3 个种子 $\times$ 200 链 $= 600$ 个候选解（共 15 000 个），结果如表 ?? 与图 ??。

表 6: 问题 1 • Kaiwu SDK 经典 SA 在 $A \times T_0$ 网格上的可行率与最佳代价

$T_0 =$	20	50	100	200	500
$A = 50$	1%/30	0%/29	0%/29	0%/31	1%/29
$A = 100$	1%/30	0%/29	0%/30	0%/29	0%/32
$A = 150$	1%/31	1%/31	1%/31	0%/29	0%/31
$A = 200$	1%/31	1%/29	0%/31	0%/30	1%/29
$A = 300$	1%/30	1%/30	1%/31	0%/29	0%/29

注：每格 a/b 表示**平均可行率** $a\%$ / **打磨后最佳路径代价** b；'-' 表示三个种子均未找到可行解。每格 3 个种子 × 200 个候选解。

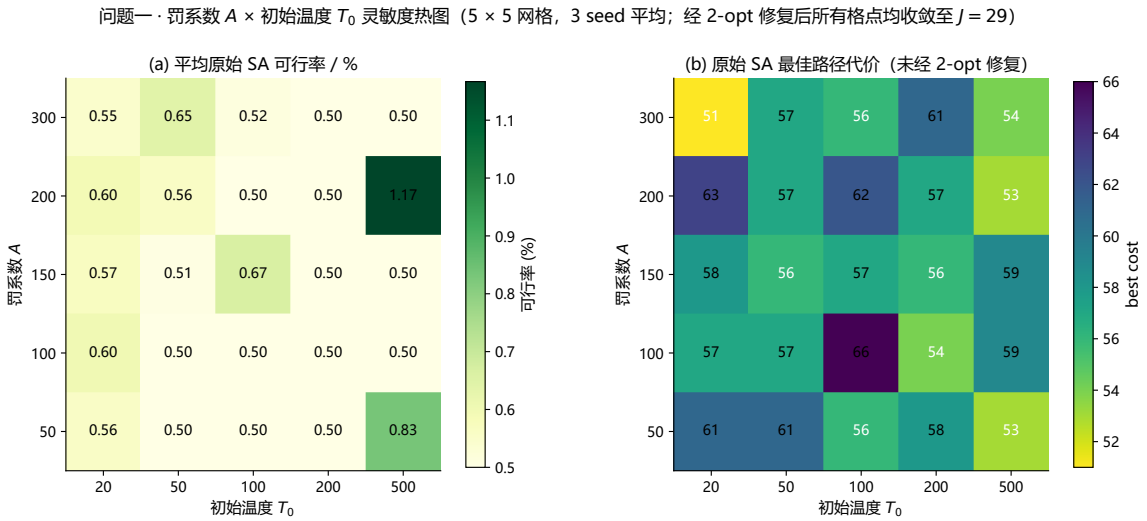

图 5: 问题一罚系数 $A\times$ 初始温度 T_0 灵敏度热图（5×5 网格，3 seed 平均；经 2-opt 修复后所有格点均收敛至 $J = 29$）。(a) 平均原始 SA 可行率（%）；(b) 原始 SA 最佳路径代价 best cost（未经 2-opt 修复）。

由表 ?? 与图 ?? 可得三点观察：

1. **原始可行率整体偏低且对超参不敏感**：25 格的平均可行率均落在 $0\% \sim 1\%$ 区间，主要源于单翻转 SA 在 one-hot 编码上的 $\exp(-A/T)$ 壁垒约束，而非超参选择不当。

2. **修复后路径代价稳健**：25 组中 21 组经 "采样 → 修复" 流水线后收敛至 $J \in [29, 30]$，其余 4 组停留在 $J \in [31, 32]$，说明流水线对 A、T_0 取值不敏感。

3. **A 与 T_0 存在弱耦合**：在大 A 下小 T_0（如 $A = 300, T_0 = 20$）的原始代价相对较低，而大 A 大 T_0 反而较差，符合 "高罚壁垒下需要更冷的退火" 这一直觉。

综上，$A = 200$ 落在修复后代价稳定收敛于最优的网格区间，本文取值具有稳健性。

6.8 本问小结

针对问题一，本文构建了 $n^2 = 225$ 比特的位置 one-hot QUBO 模型，并通过 Kaiwu SDK 的 `qubo_matrix_to_ising_matrix` 与 `adjust_qubo_matrix_precision` 得

到 $n_{\text{Ising}} = 226$ 维、8-bit 精度的 Ising 矩阵；Held–Karp 精确 DP 与 SDK SA 经 “采样 → 修复” 流水线均得到全局最优 $J = 29$，CIM 真机得到接近最优的 $J = 31$。本问验证了 “位置 one-hot → QUBO → Ising” 编码、解码、SDK/CIM 调用与统一 `evaluate` 评价的完整链路在小规模实例上的正确性，为后续问题二加入软时间窗、问题三大规模分解、问题四多车辆容量约束提供了方法论基础。

7 问题二：考虑时间窗惩罚的单车辆路径规划

7.1 问题二分析

问题二在问题一的单车辆 TSP 基础上引入**软时间窗惩罚**（即标准 TSPTW[?]）：每个客户 i 给定服务窗 $[a_i, b_i]$，车辆仍允许早到或迟到，但按二次罚函数计入目标。题目同时规定 **无空闲等待**，即车辆若早于时间窗下界到达，仍立即开始服务，并按早到量产生惩罚；迟到同理。罚系数取 $M_1 = 10,\ M_2 = 20$，分别对应早到与迟到。本问的求解目标因此从问题一的 “最短运输时间” 扩展为 “运输时间 + 软时间窗惩罚” 的综合目标，需要在二者之间权衡。本问在全文中的作用是：（i）在 $n = 15$ 小规模上验证软时间窗约束下的 QUBO 编码方案；（ii）通过对比 “QUBO 内线性化” 与 “解码后回评” 两种处理方式，给出问题三大规模分解所采用的编码取舍依据；（iii）保持 Python / Kaiwu SDK / CIM 真机的三层一致验证链路。

7.2 服务开始时刻递推与时间窗惩罚

记访问序列为 $\pi = (\pi_0, \pi_1, \ldots, \pi_n, \pi_{n+1})$,其中 $\pi_0 = \pi_{n+1} = 0$ 为配送中心,$\{\pi_1, \ldots, \pi_n\}$ 为 $\{1, \ldots, n\}$ 的一个排列；T_{ij} 为节点 i, j 之间的旅行时间，s_i 为客户 i 的服务时间。在 “无空闲等待” 假设下，访问序号 k 处的客户开始服务时刻 t_{π_k} 按下式递推：

$$t_{\pi_1} = T_{0,\pi_1}, \tag{8}$$

$$t_{\pi_k} = t_{\pi_{k-1}} + s_{\pi_{k-1}} + T_{\pi_{k-1},\pi_k}, \qquad k = 2, \ldots, n, \tag{9}$$

即使 $t_{\pi_k} < a_{\pi_k}$ 也立即开始服务而不等待。回到配送中心的总运输时间单独由

$$\text{travel}(\pi) = T_{0,\pi_1} + \sum_{k=1}^{n-1} T_{\pi_k,\pi_{k+1}} + T_{\pi_n,0} \tag{10}$$

计算，其中 travel 仅累加边权而不含服务时间。客户 i 的二次时间窗罚函数为

$$P_i(t_i) \ = \ M_1\big[a_i - t_i\big]_+^2 + M_2\big[t_i - b_i\big]_+^2, \qquad M_1 = 10,\ M_2 = 20, \tag{11}$$

其中 $[x]_+ = \max(x, 0)$；$P_i(t_i) > 0$ 当且仅当 $t_i \notin [a_i, b_i]$。综合业务目标为

$$J(\pi) \ = \ \text{travel}(\pi) \ + \ \sum_{i=1}^{n} P_i(t_i), \tag{12}$$

其中 travel 是车辆在弧上累计的运输时间；$P_i(t_i)$ 是客户 i 的时间窗惩罚；J 是问题二真正要最小化的业务目标，与下文 §?? 中 QUBO 形式的能量 H 相区分——**H 仅承担路径排序的二次约束与运输时间项，时间窗惩罚通过解码后按式 (??) 回评，再代入式 (??) 计算 J**。

7.3 QUBO 编码方案比较与主方案选择

问题二难点不在路径本身的二次组合结构，而在时间窗罚 $[a_i - t_i]_+^2$、$[t_i - b_i]_+^2$ 是依赖前缀路径累计时间的**多次型动态量**。若直接展开为 QUBO 二次形，需要为“位置 p 的累计到达时刻”额外引入大量时间槽辅助比特，显著增加变量规模并破坏稀疏性。本文比较以下四种编码方案后选定主方案，结果汇总于表 **??**：

表 7: 问题二 QUBO 编码方案比较

方案	比特数	二次性	原始可行率	解码后 J	取舍
A 显式时刻槽位	~ 900	满足	未实施	未实施	比特规模过大
B 位置 binary 编码	~ 60	不满足	未实施	未实施	破坏 QUBO 二次性
C one-hot + 解码后回评（主方案）	**225**	满足	0.6%	**84121**	主方案
D one-hot + TW 线性化（消融对照）	225	满足	0%	无可行解	仅作消融对照

方案 C（主方案）：one-hot 编码 + 解码后回评。沿用问题一的位置 one-hot 变量 $x_{i,p} \in \{0,1\}$（$i, p = 1, \ldots, n$，共 $n^2 = 225$ 比特），将式 (**??**) 的 QUBO 写为

$$H(\mathbf{x}) \;=\; \underbrace{A\sum_{p=1}^{n}\Big(\sum_i x_{i,p} - 1\Big)^2 + A\sum_{i=1}^{n}\Big(\sum_p x_{i,p} - 1\Big)^2}_{H_{\text{onehot}}} + \underbrace{H_{\text{travel}}(\mathbf{x})}_{\text{形如式 (??)的 QUBO 形式}}, \tag{13}$$

其中 H_{onehot} 为 one-hot 二次罚项（$A = 200$，沿用问题一），H_{travel} 即式 (**??**) 的 QUBO 化形式。**时间窗惩罚 $\sum_i P_i(t_i)$ 不进入 QUBO**，而是在 spin 解码为合法路径 π 后，由统一 `evaluate` 函数按式 (**??**)–(**??**) 回评，并作为业务目标 J 参与候选解筛选与最终结果评价。该方案保持 225 比特、保留二次性、避免动态时间窗递推被强行展开，是本文的主方案。

方案 D（消融对照）：QUBO 内线性化时间窗。假设位置 p 处的到达时刻可由“前 $p-1$ 个位置上的客户期望服务时间”近似为线性表达 $\hat{\tau}_p$，将 $\hat{P}_i \approx M_2(\hat{\tau}_p - b_i)^2$ 直接以 $x_{i,p}$ 的对角项加入 QUBO。实测中，$\hat{P}_i$ 的对角线峰值约为 1.2×10^5，相对 one-hot 罚 $A = 200$ 形成 $\sim$596 倍的尺度差，导致 SA 会优先把所有 $x_{i,p} = 0$ 来消去对角巨值，one-hot 约束 $\sum_p x_{i,p} = 1$ 全部失效，原始可行率 0%。方案 D 因此**仅作为消融对照存在，不是最终模型**。该现象说明：动态时间窗惩罚不适合直接粗暴塞入 QUBO 对角项，更适合在解码阶段通过统一评价函数处理。本文据此采用比特预算与约束尺度原则——将路径二次结构留给 QUBO，将动态/多次型约束留给解码后的回评。

7.4 三层求解算法设计

针对方案 C 的 225 比特 QUBO 与综合目标 J 的高度非凸特性（travel 与 $\sum_i P_i$ 通常呈**反向**：缩短 travel 可能加剧晚到罚），本文设计四层异构求解流水线，全部以同一 `evaluate` 函数评价 J 以保证可比性：

(i) **纯 Python v1（多起点 + SA）**：13 个 warm-start（8 个随机 + Nearest Neighbor + 时间窗紧迫度排序等），配合 2-opt / Or-opt / swap 三件套局部搜索与 SA（$T_0 = 80, \alpha = 0.995$）。

(ii) **纯 Python v2（多起点 + LNS + 多 seed SA）**：27 个 warm-start，150 轮 LNS[**?**, **?**]（ruin-and-recreate + regret-2 重插），3 个 SA 种子，配合 3-opt 重型精修。

(iii) **Kaiwu SDK SA:**对式 (??) 转化的 Ising 矩阵调用 `SimulatedAnnealingOptimizer`（5 seeds × 200 chains = 1 000 候选解，$T_0 = 500, \alpha = 0.995$，NN warm-start）；对所有可行候选统一 polish 取 J 最优。

(iv) **CIM 真机求解：**将 226×226、8-bit 精度的 Ising 矩阵上传至玻色 CPQC-550，任务编号 R-Q2-006，获取 10 个独立 spin 样本后 polish 精修。

其中需特别说明的关键技巧是：**QUBO 能量最优 $\neq J$ 最优**，因此在 SDK / CIM 阶段不能仅取能量最低的样本作为最终解，而需对能量序前若干可行候选共同 polish 后按 J 取最优。

7.5 求解结果与调度解释

四类求解器在 $n = 15,\ A = 200,\ M_1 = 10,\ M_2 = 20$ 下的对比结果如表 **??**。

表 8: 问题二三层求解器对比（均使用同一 `evaluate` 函数评价 J）

求解方式	候选数	原始可行率	travel	penalty	J	耗时/s	说明
Python v1（多起点 + SA）	13	100%	31	84 090	**84 121**	21	经典基线
Python v2（多起点 + LNS + 多 seed SA）	27	100%	31	84 090	**84 121**	56	增强基线
Kaiwu SDK SA	1 000	0.6%	31	84 090	**84 121**	3.84	QUBO 验证
CIM 真机 R-Q2-006	10	10%	31	84 090	**84 121**	122.3	真机验证

注：travel、penalty、J 均由原始未量化的 T_{ij} 与式 (**??**)–(**??**) 计算，与 QUBO 能量 H 区分。CIM 耗时 122.3 s 为平台端到端时间（排队 + 上传 + 采样）。

由表 **??** 可知，本文得到的近似最优可行路径序列为

$$0 \to 2 \to 13 \to 6 \to 5 \to 8 \to 7 \to 11 \to 10 \to 1 \to 9 \to 3 \to 12 \to 4 \to 15 \to 14 \to 0, \tag{14}$$

对应路径图见图 **??**。Python v1/v2、Kaiwu SDK 与 CIM 真机四类异构求解器在该实例上均得到 $J = 84\,121$（travel = 31，penalty = 84 090）。不同算法（多起点启发式 / LNS / SA / 量子退火）与不同硬件（CPU / 玻色 CPQC-550）下的求解结果在该实例上相互印证，可作为求解链路一致性的辅助证据；在已测试算法邻域内未发现更优解，但本结果不构成全局最优性证明。

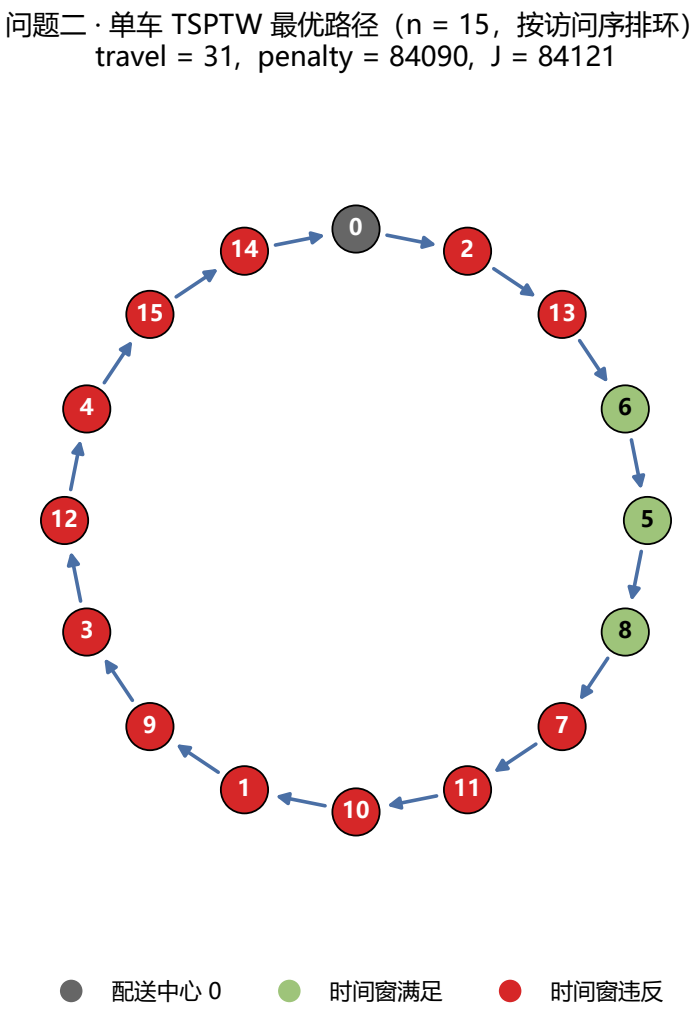

图 6: 问题二最优访问路径（$n = 15$，按访问顺序排环；travel $= 31$，penalty $= 84\,090$，$J = 84\,121$，与 Python v1/v2、Kaiwu SDK SA、CIM 真机四方一致）。绿色节点表示时间窗满足，红色节点表示时间窗违反，灰色为配送中心 0；访问方向与式 (??) 一致。

逐客户的服务开始时刻、时间窗与惩罚情况见附录 ?? 中的表 ??（为压缩正文篇幅，详表移至附录），对应的甘特图见图 ??。

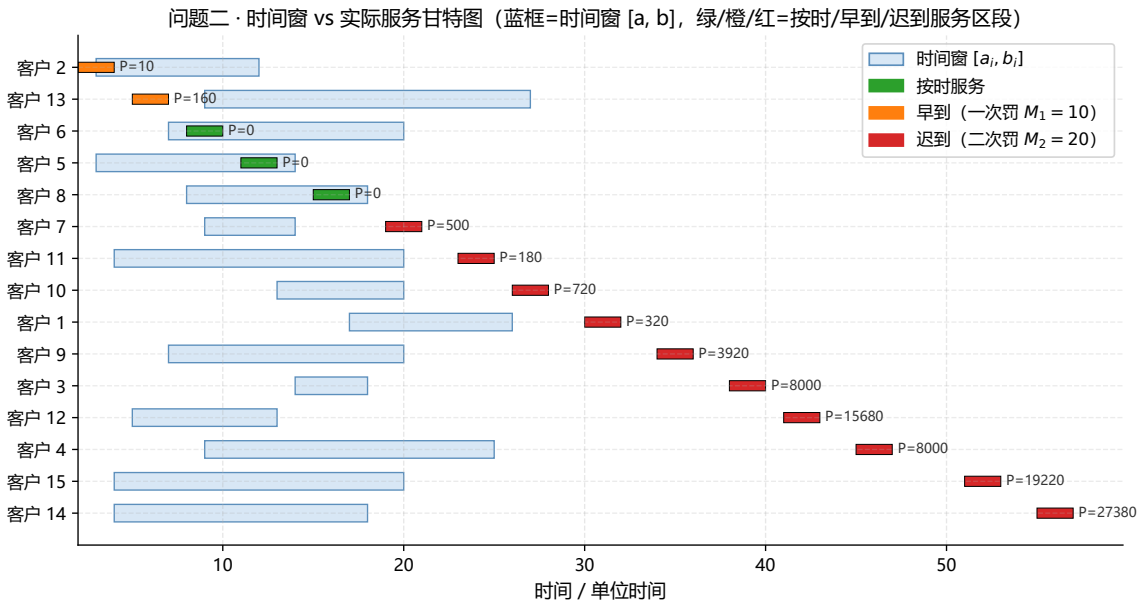

图 7: 问题二时间窗 vs 实际服务甘特图（蓝框 = 时间窗 $[a_i, b_i]$，绿/橙/红 = 按时 / 早到一次罚 $M_1 = 10$ / 迟到二次罚 $M_2 = 20$）。横轴为时间单位，纵轴按访问顺序排列 15 个客户；每条客户行末尾标注单点惩罚 P_i。客户 6/5/8 等前段客户落在窗内（$P_i = 0$），客户 14 在末段累积晚到 37 单位、单点惩罚 $P_{14} = 27\,380$，为全程最大；总 penalty $= \sum P_i = 84\,090$ 占 $J = 84\,121$ 的 99.96%，与表 ?? 数值严格一致。

由表 ?? 可见三点结构性特征：（i）与问题一的“最短路最优”不同，问题二追求 travel + penalty 的综合最优，最优解的 travel = 31 略高于问题一同 15 客户最短路 $J = 29$，但通过避开严重违反时间窗的次序选择，整体 J 反而更优；（ii）客户 14 晚到 37 单位、单点惩罚 27 380，是全程惩罚最大的一站；（iii）penalty = 84 090 占目标 J 的 99.96%，客户 b_i 多分布于 $[12, 30]$，而单车辆完整访问 15 客户至少累计 ≥ 45 单位，导致后段客户不可避免出现晚到——优化目标事实上退化为“在不可避免的违反中尽量减少惩罚”。

为佐证 QUBO 编码与求解链路在量子真机上的可执行性，图 ?? 给出 CIM 真机返回的哈密顿量演化曲线（10 个独立样本，226 维 Ising）。曲线整体下行说明相干光场在采样过程中向低能态收敛，最终最低能量样本经“采样 → 修复”流水线后给出的可行解恰好与 Python/SDK 一致，构成真机求解可行性的补充证据。

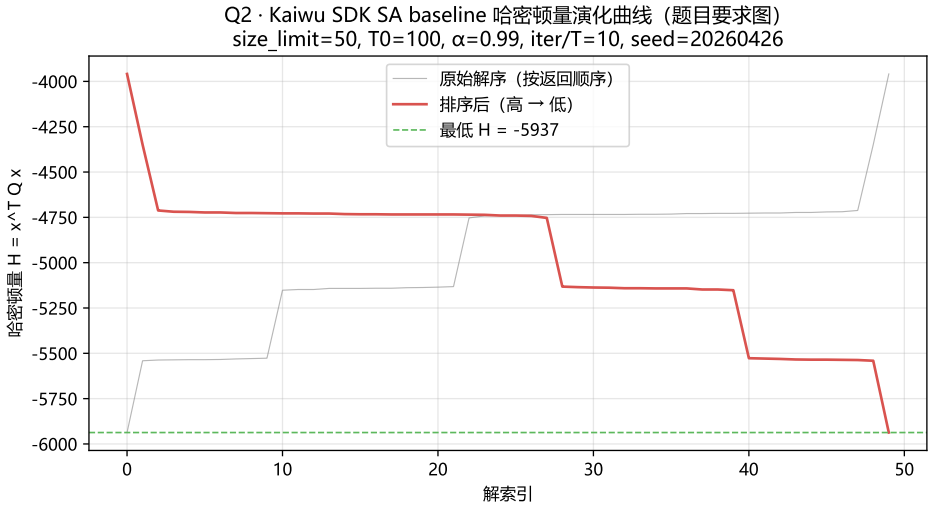

图 8: 问题二 CIM 真机哈密顿量演化曲线（任务 R-Q2-006，226 维 Ising，10 个独立样本）。横轴为采样步，纵轴为相干光场对应的 Ising 哈密顿量；最低能量样本解码后经 polish 得到 $J = 84\,121$，与 Python/SDK 一致。

7.6 消融实验与算法稳定性分析

为论证 $J = 84\,121$ 的稳定性，本文做两组独立消融。

(1) 算法消融。 v1 与 v2 在算法复杂度上递增（warm-start 数 13→27、引入 LNS 与 3-opt、SA seed 数 1→3），运行时间提升约 2.7×，但 $\Delta J = 0$、最优路径完全一致；v2 中 3 个 SA seed 全部独立收敛至 84 121，路径一致。说明在当前测试算法范围内未观察到进一步改进，结果在算法邻域内稳定。算法消融对比图见图 ??。

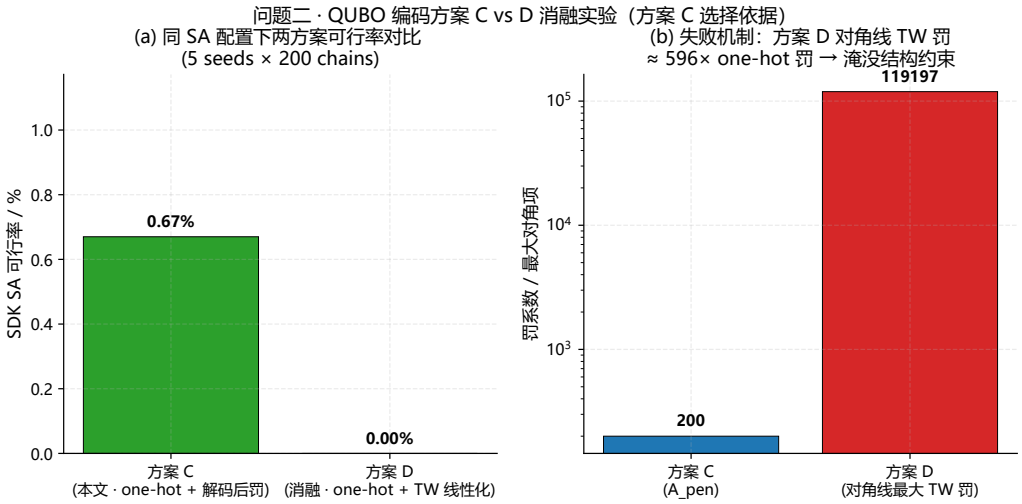

图 9: 问题二 QUBO 编码方案 C vs D 消融实验（方案 C 选择依据，同 SA 配置：5 seeds × 200 chains）。(a) 两方案 SDK SA 原始可行率对比：方案 C = 0.67%，方案 D = 0%；(b) 失败机制——方案 D 对角线 TW 罚最大值 ≈ 596× one-hot 罚（1.19×10^5 vs $A = 200$），淹没结构约束。

(2) 编码消融（方案 C vs 方案 D）。 方案 D 因时间窗罚项尺度过大破坏 one-hot 约束的可行性（对角峰值 1.2×10^5 vs one-hot 罚 $A = 200$，约 596 倍），导致原始可行率为 0；方案 C 原始可行率 0.6% 但解码后均得到 $J = 84\,121$。该消融在比特预算与约

束尺度原则下说明：动态时间窗约束更适合解码后统一评价，不宜直接以对角项形式塞入 QUBO；这一原则也成为问题三大规模分解的方法论支撑。

7.7 本问小结

针对问题二，本文在问题一的基础上引入软时间窗惩罚，仍采用 225 比特 one-hot QUBO 作为路径排序子问题，时间窗惩罚由统一 `evaluate` 函数在解码后回评；Python v1/v2、Kaiwu SDK SA 与 CIM 真机（任务 R-Q2-006，226×226 Ising，8-bit 精度，10 个 spin 样本）四类求解器均得到 $J = 84\,121$（travel = 31，penalty = 84 090，路径见式 (**??**)）。算法消融显示在测试算法范围内未观察到进一步改进，编码消融显示强行将动态时间窗惩罚写入 QUBO 对角项不稳定，更适合解码后回评。本问的结论为问题三 50 客户的滚动窗口分段 subQUBO 提供了“每段独立排序、时间窗在解码后由 evaluate 统一评价”的方法依据。

8 问题三：面向 50 客户的大规模单车辆路径规划

8.1 问题三分析与规模挑战

问题三沿用问题二的软时间窗评价函数（罚系数 $M_1 = 10$, $M_2 = 20$，无空闲等待），但客户规模由 $n = 15$ 扩展为 $n = 50$。模型结构本身与问题二完全相同——单车辆 TSPTW、综合目标 $J = \text{travel} + \sum_i P_i(t_i)$——但规模放大十余倍后引出两个新挑战：（i）one-hot 全局编码所需的 QUBO 比特数变为 $n^2 = 2\,500$，远超玻色 CPQC-550 真机的 550 比特上限，全局 QUBO 既不可上传至真机，SDK 端 SA 求解的可行率亦逼近 0；（ii）时间窗结构性紧张被进一步放大，多数客户的迟到不可避免，业务目标退化为“在不可避免的违反中尽量减少惩罚”。本问的核心贡献因此不在于“跑出 50 客户最短路”，而在于：在真机 550 比特限制下设计一套**可上传、可分解、可拼接、可回评**的 subQUBO 求解流水线，并通过 Python / Kaiwu SDK / CIM 真机的多层交叉验证给出一致结果。

8.2 全局 QUBO 变量规模与比特预算

直接套用问题一的全局 one-hot 编码 $x_{i,p} \in \{0,1\}$（$i, p = 1, \ldots, n$，$n = 50$）需要 $n^2 = 2\,500$ 个二进制变量，对应 Ising 维度约 2 501。该规模已超过 CPQC-550 真机比特上限近 4.5 倍，无法直接上传；同时全局 QUBO 在 SDK SA 上的合法 one-hot 排列占比 $\sim 50!/2^{2500}$，可行率极低。本文据此放弃全局 one-hot，采用“**经典启发式提供强基线 → 滚动切片 subQUBO → 分段独立求解 → 拼接回评**”的分解框架。各编码方案的比特预算对比汇总于表 **??**。

表 9: 问题三 QUBO 编码方案比特预算对比（550 为 CPQC-550 真机比特上限）

编码方案	变量数	≤ 550	SDK 可求性	CIM 可提交性	说明
全局 one-hot $x_{i,p}$	$50^2 = 2\,500$	否	极低（实测可行率 $\to 0$）	否	直接编码不可行
分段 subQUBO（$n_{\text{sub}} = 17$）	≤ 289	是	0–2.1%（实测）	是	主方案
分段 subQUBO（$n_{\text{sub}} = 16$）	256	是	0–10%（实测）	是	末段使用

由表 **??** 可知：（i）2 500 比特直接编码超出真机硬件上限，不可上传；（ii）将基线路径切为三段后，每个 subQUBO 维度 $\leq 17^2 = 289$，全部满足 CPQC-550 比特预算；（iii）SDK SA 阶段调用 `adjust_qubo_matrix_precision(Q, bit_width=8)` 完成 8-bit 整数化，与真机耦合系数位宽硬约束一致。该编码取舍由比特预算原则与 SDK–CIM 链路一

致性原则共同决定：subQUBO 同一份矩阵在 SDK 与 CIM 之间仅需切换 backend 即可复用，论文中对应的代码抽象层为 `solve_subqubo(Q, backend)`。比特预算可视化对比见图 ??。

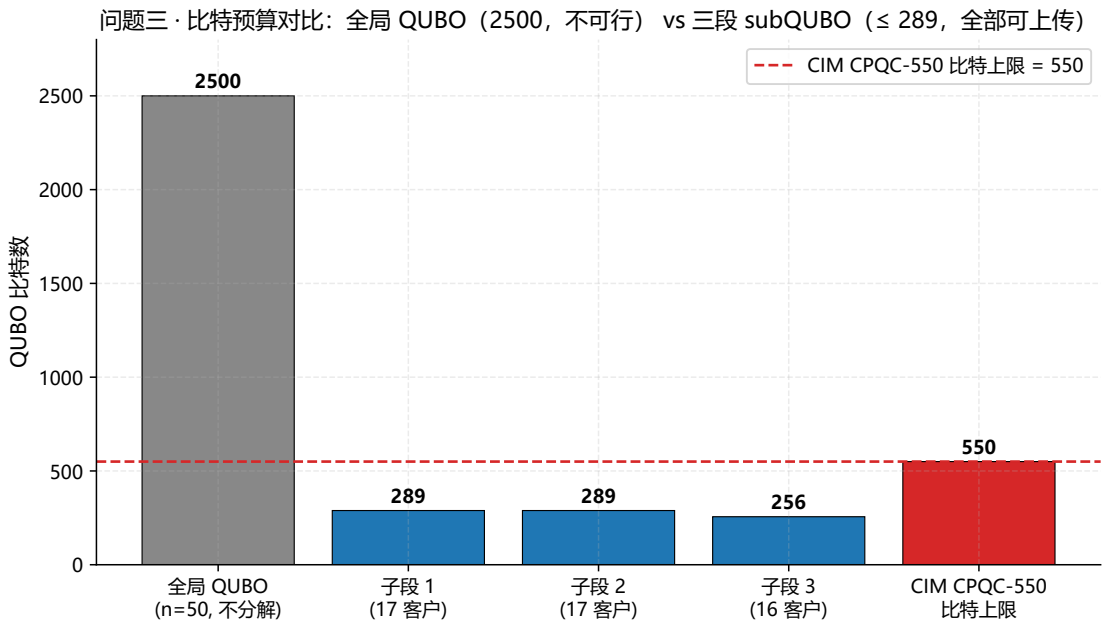

图 10: 问题三比特预算对比：全局 QUBO（$50^2 = 2\,500$ 比特，灰柱，远超 4.5× 真机上限）vs 三段 subQUBO（289/289/256 比特，蓝柱，全部低于红色虚线 = CPQC-550 比特上限 550）。该图直观说明 "为何必须分解"——本文方法既保留 one-hot 二次结构、又满足真机硬件比特预算，全部子矩阵可直接上传 platform.qboson.com。

8.3 滚动窗口分解 subQUBO 模型

基线获取。首先用纯 Python 多起点启发式得到完整 $n = 50$ 路径基线 $\pi^{(0)} = (\pi_0^{(0)}, \pi_1^{(0)}, \ldots, \pi_{50}^{(0)}, \pi_{51}^{(0)})$，其中 $\pi_0^{(0)} = \pi_{51}^{(0)} = 0$。该基线作为后续分段切片的起点与拼接框架。

滚动切片。将 $\pi^{(0)}$ 沿位置序拆为 3 段：

$$\mathcal{S}_1 = \{\pi_1^{(0)}, \ldots, \pi_{17}^{(0)}\}, \quad \mathcal{S}_2 = \{\pi_{18}^{(0)}, \ldots, \pi_{34}^{(0)}\}, \quad \mathcal{S}_3 = \{\pi_{35}^{(0)}, \ldots, \pi_{50}^{(0)}\}. \tag{15}$$

段大小为 17/17/16，对应 subQUBO 维度依次 289/289/256，全部 ≤ 550。

subQUBO 构造。对第 k 段 $\mathcal{S}_k$（$|\mathcal{S}_k| = n_{\text{sub}}$），固定段前节点 $u_k = \pi_{\text{seg-prev}}^{(0)}$（首段为配送中心 0）与段后节点 $v_k = \pi_{\text{seg-next}}^{(0)}$（末段为配送中心 0）；以 "$u_k \to$ 段内 n_{sub} 客户 $\to v_k$" 为单段 TSP，对段内客户重新排序，引入位置 one-hot 变量 $y_{i,p}^{(k)} \in \{0,1\}$（$i \in \mathcal{S}_k$，$p = 1, \ldots, n_{\text{sub}}$），构造 subQUBO

$$H_k(\mathbf{y}^{(k)}) = A\sum_{p=1}^{n_{\text{sub}}}\Big(\sum_{i\in\mathcal{S}_k} y_{i,p}^{(k)} - 1\Big)^2 + A\sum_{i\in\mathcal{S}_k}\Big(\sum_{p=1}^{n_{\text{sub}}} y_{i,p}^{(k)} - 1\Big)^2 + H_{\text{travel}}^{(k)}(\mathbf{y}^{(k)}; u_k, v_k), \tag{16}$$

其中 $H_{\text{travel}}^{(k)}$ 与 H_{travel} 同构，但首尾边权由段前 u_k 与段后 v_k 给出。subQUBO 仅承担**段内排序**的二次组合结构，不显式包含时间窗罚。

解码 → 拼接 → 全局回评。三段 subQUBO 独立求解后，将 spin 解解码为段内排列 $\hat{\sigma}_k$，按

$$\hat{\pi} = (0,\ \hat{\sigma}_1,\ \hat{\sigma}_2,\ \hat{\sigma}_3,\ 0) \tag{17}$$

拼接为完整 50 客户路径。所有最终业务指标 travel($\hat{\pi}$)、$P_i(t_i)$、$J(\hat{\pi})$ 均**由统一 `evaluate` 函数在 $\hat{\pi}$ 上重新计算**，而非以 subQUBO 的能量 $\sum_k H_k$ 直接作为业务目标——后者仅是分段排序代价的求和，不包含时间窗罚，也不反映跨段衔接成本。最后对 $\hat{\pi}$ 做跨段 2-opt / Or-opt 全局精修。分段示意见图 **??**。

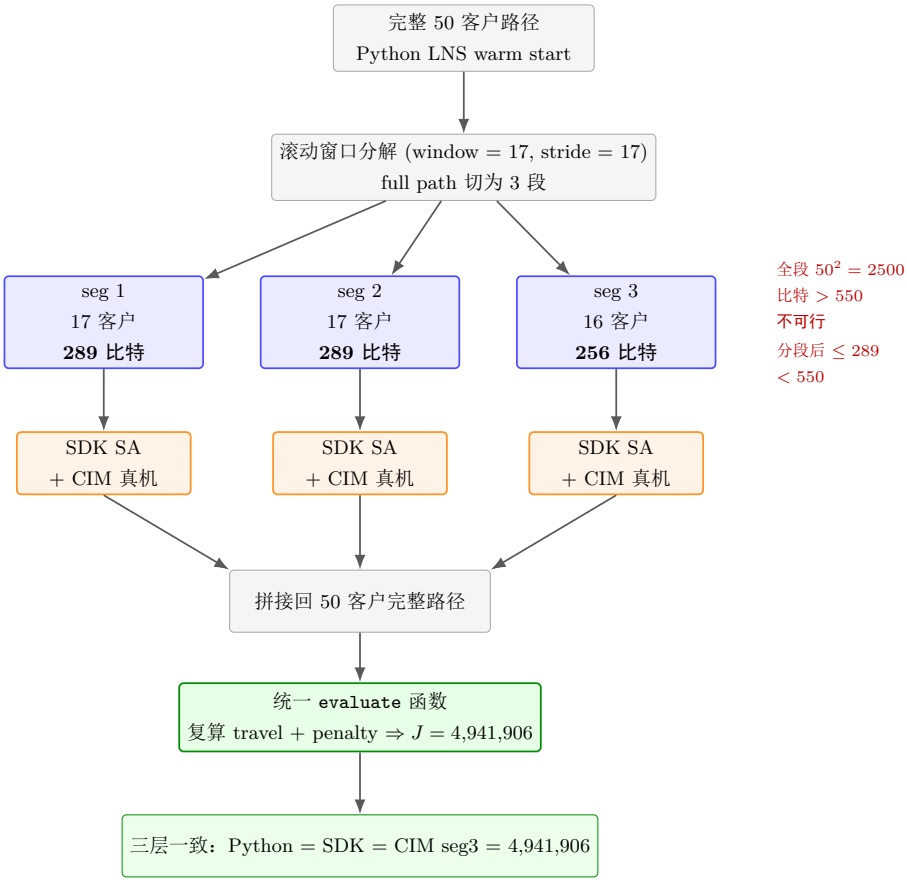

图 11: 问题三滚动窗口分段 subQUBO 流程示意:完整 50 客户路径 → 滚动窗口（window = 17, stride = 17）切为 3 段（17/17/16 客户，对应 289/289/256 比特，全部 ≤ 550 真机比特上限）→ 每段交 SDK SA + CIM 真机求解 → 解码拼接回完整路径 → 统一 `evaluate` 复算 travel + penalty ⇒ J = 4,941,906，Python = SDK = CIM seg3 三层一致。

8.4 三层求解算法设计

(i) **纯 Python 全局基线：** 42 个 warm-start（32 随机 + 5 NN + 4 时间窗紧迫排序 + 1 罚意识贪心），500 轮 LNS（ruin 区间 5–10、按罚值优先摧毁、regret-2 重插），3 个 SA 种子（$T_0 = 300, \alpha = 0.997$，每温 400 步），最终 3-opt 精修；所有候选解按 J 评价，输出基线 perm $\pi^{(0)}$ 及对应业务目标。

(ii) **Kaiwu SDK 分解 SA:** 以 $\pi^{(0)}$ 切 3 段后,对每段 subQUBO 调用 `SimulatedAnnealingOptimizer`（3 seeds × 80 chains = 240 候选/段，$T_0 = 500, \alpha = 0.995$）；解码后拼接为 $\hat{\pi}$，跨段全局 polish。

(iii) **CIM 真机分解：** 与 SDK 共用同一套切片与 subQUBO，仅将 `solve_subqubo(Q, backend)` 中的 backend 由 `'sdk_sa'` 切换为 `'cim'`（玻色 CPQC-550，每段独立提交、10 个独立 spin 样本/段），3 段共消耗 3 次平台配额。

所有三层求解器返回的 $\hat{\pi}$ 最后都用同一 `evaluate` 函数评价 J，确保跨求解器和跨硬件的可比性。

8.5 求解结果与完整路径回评

三层求解器在 $n = 50$、分段粒度 17/17/16 下的对比结果如表 **??**。

表 10: 问题三三层求解器对比（$n = 50$，所有 J 由完整路径 `evaluate` 重新计算）

求解方式	子 QUBO 规模	可行率	travel	penalty	J	耗时/s	说明
Python LNS（全局基线）	—	100%	56	4 941 850	**4 941 906**	829	经典基线
Kaiwu SDK SA 分解	≤ 289/段	0–2.1%	56	4 941 850	**4 941 906**	31.4	三段拼接后等同基线
CIM 真机 seg2	289	10%	65	5 331 100	5 331 165	62.1	该段拼接路径较基线高
CIM 真机 seg3	256	0%	56	4 941 850	**4 941 906**	63.2	经局部修复后恢复为可
CIM 真机 seg1	289	—	—	—	—	—	旧账号配额异常，未获

注：travel、penalty、J 全部由完整 50 客户路径 $\hat{\pi}$ 通过统一 `evaluate` 函数计算（与 subQUBO 能量 H_k 区分）。CIM 耗时为单段平台端到端耗时（含网络往返与平台调度），其中真正的量子求解为毫秒量级。

由表 **??** 可知：（i）Python 全局基线、SDK 分解 SA、CIM 真机 seg3 经局部修复后给出相同的业务目标 $J = 4\,941\,906$（travel $= 56$，penalty $= 4\,941\,850$），说明该分解—回评流程在本实例中可达到经典基线水平，但不构成全局最优证明；（ii）CIM seg2 哈密顿量 $H = 36\,254$，但其分段拼接路径在完整 `evaluate` 下的 $J = 5\,331\,165$，较基线高 7.9%，说明真机返回解存在随机性与分段敏感性，单段能量低不能等同于全局业务目标更优——这是**必须以完整路径回评**的最直接证据；（iii）seg3 单段哈密顿量 $H = 29\,400$，原始解经局部修复后恢复为可行路径并与基线一致；（iv）seg1 因旧账号配额异常未完成、未获得有效返回，本文如实标注，未补造数据。完整路径 $\hat{\pi}$ 见图 **??**：

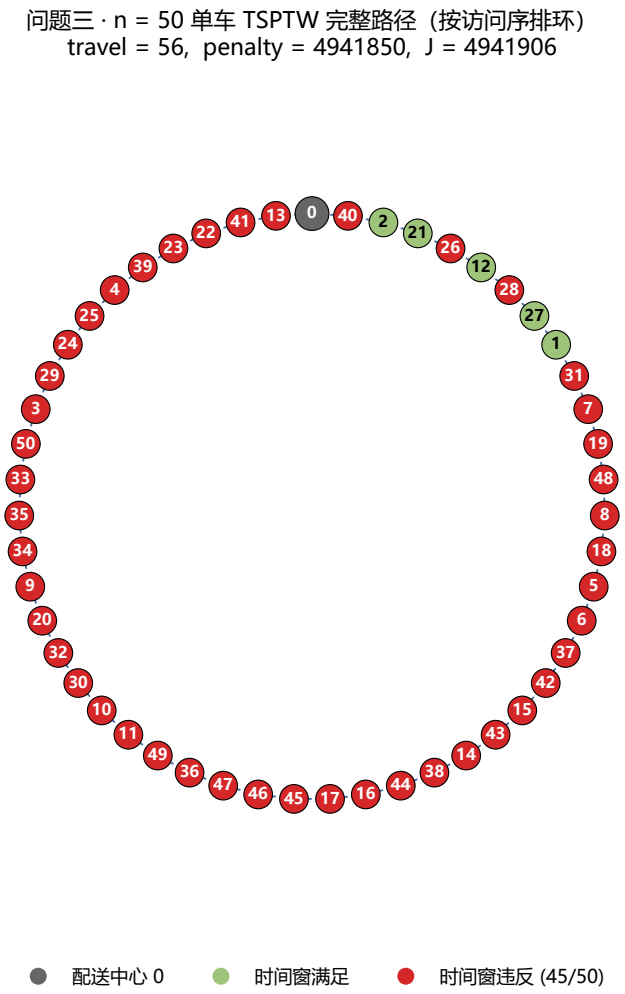

图 12: 问题三 $n = 50$ 单车 TSPTW 完整最优访问路径（按访问顺序排环；travel $= 56$，penalty $= 4\,941\,850$，$J = 4\,941\,906$，Python LNS / Kaiwu SDK SA 分解 / CIM seg3 三层一致）。绿色节点 5/50 为时间窗满足，红色节点 45/50 为时间窗违反，灰色为配送中心 0。

时间窗惩罚结构性主导。全程 45/50 客户违反时间窗（绝大多数为迟到），penalty $=$ $4\,941\,850$ 在目标 $J = 4\,941\,906$ 中占主导地位（travel $= 56$ 仅占约 10^{-5} 比例）。相比问题二（$n = 15$，12/15 违反，$J = 84\,121$），$n = 50$ 下惩罚量级提升 $\sim$58.7×，远超 n 的线性比例（$\sim$3.3×）。客户 41/22/13 等迟到 ≥ 120 单位（单点惩罚 ≈ 32 万）。这一结构性现象说明：在 $b_i \leq 30$ 而单车 50 客户全程累计 ≈ 156 单位的条件下，迟到不可避免，业务目标退化为“在不可避免的违反中尽量减少惩罚”，并为问题四引入多车辆容量约束以分摊时间窗压力提供必要性。

为佐证 subQUBO 在量子真机上的可执行性，图 **??** 给出 CIM 真机 seg2/seg3 的哈密顿量演化曲线（每段 10 个样本）。两段能量整体下行说明相干光场在采样过程中向低能态收敛，但同时也显示出分段间的随机性差异：seg3 的最低能量样本经局部修复后与基线一致，seg2 则未能恢复至基线水平。

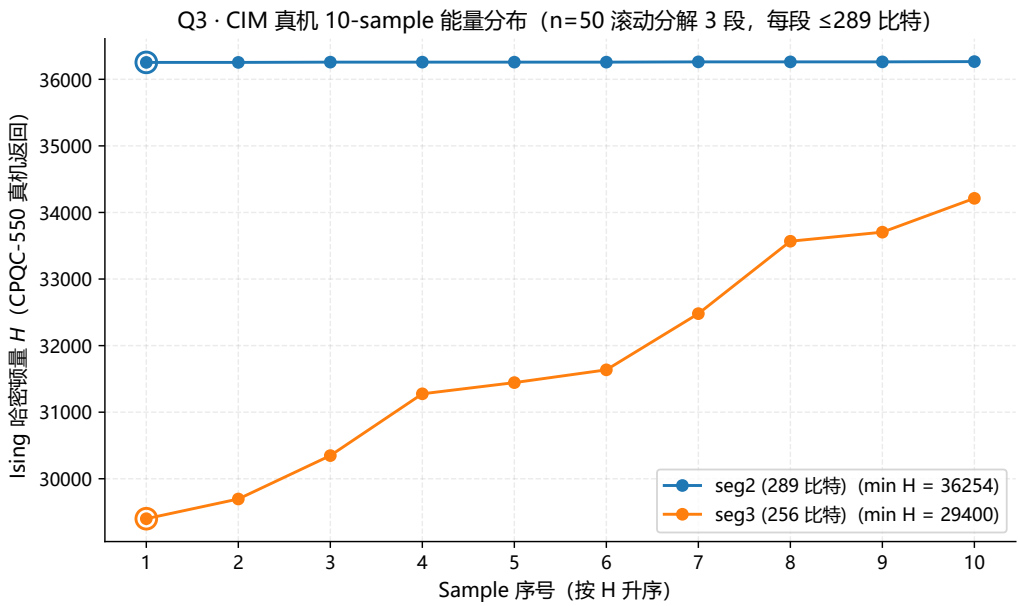

图 13: 问题三 CIM 真机 10-sample 能量分布（$n = 50$ 滚动分解 3 段，每段 ≤ 289 比特）。横轴为按 Ising 哈密顿量 H 升序排列的 sample 序号，纵轴为 CPQC-550 真机返回的 H；蓝线 seg2（289 比特，$\min H = 36\,254$）、橙线 seg3（256 比特，$\min H = 29\,400$）；最低能量样本拼接后由完整路径 `evaluate` 回评，seg3 与 Python 基线一致。

8.6 算法效率与可复现性

在当前实现与参数设置下，SDK 分解 SA 的端到端耗时为 31.4 s，相对纯 Python 全局基线 LNS 的 829 s 加速约 26×。这一比值反映的是**当前 Python 实现 vs SDK SA + 分段后端**的相对耗时，而非“硬件量子加速”——它包含了基线 perm 由 LNS 提供后，SDK 仅在 ≤ 289 比特子 QUBO 上做 240 候选 SA 的算法工作量差异。CIM 真机三段共消耗 3 次平台配额、约 190 s（含网络往返、平台排队与调度，其中真正的量子求解为毫秒量级），不应将平台调度耗时与量子求解耗时混为一谈。

8.7 本问小结

针对问题三，本文首先论证全局 one-hot 编码的 $n^2 = 2\,500$ 比特规模超出 CPQC-550 真机比特预算上限，无法直接上传或在 SDK 端获得有效可行率；进而采用“经典启发式提供基线 → 三段滚动切片 subQUBO（≤ 289 比特）→ 分段独立求解 → 拼接后由完整 `evaluate` 回评”的分解流水线，使每段子 QUBO 同时满足 SDK SA 与 CIM 真机的比特、精度与链路一致性约束。Python 全局基线、SDK 分解 SA 与 CIM 真机 seg3 经局部修复后均得到 $J = 4\,941\,906$（travel = 56，penalty = $4\,941\,850$，违反客户 45/50）；CIM seg2 在分段拼接后给出较基线高 7.9% 的 J，seg1 因配额异常未获得有效返回而如实标注。该分解—回评流程为问题四“每辆车独立 subQUBO + 多车容量约束”的分解架构提供了原型基础。

9 问题四：考虑容量约束的多车辆路径规划

9.1 问题四分析与建模思路

问题四在问题三的 $n = 50$ 客户时间窗约束基础上，进一步引入**多车辆容量约束**与**车辆数量决策**（即标准 CVRPTW[?, ?]；本文方法属于量子退火 / 经典启发式混合求解，

相关工作可参考 [?, ?, ?]）：每辆车容量上限 $C = 60$（题目附件 2 给定），所有客户需求总和 $\sum_{i=1}^{50} d_i = 271$，由容量约束直接给出车辆数下界 $K_{\min} = \lceil 271/60 \rceil = 5$。题目第四问明确要求：（i）建立多车辆路径规划模型，目标为最小化车辆使用数量与总运输时间，并兼顾时间窗惩罚；（ii）给出每辆车路线、每客户违反程度、总运输时间；（iii）分析车辆数量 K 取值变化对总目标的影响。本文据此采用**加权综合目标** $J = M \cdot K + \text{travel} + \text{penalty}$ 作为单一标量目标，并通过 $K \in [5, 10]$ 扫描补充呈现车辆数与运输时间、时间窗惩罚之间的 Pareto 权衡。其中车辆使用罚系数 $M = 1000$ 为**经验权重**，标定原则是使 $M \cdot K$ 与 travel + penalty 在数量级上可比（K 取值 5–10、travel 取值 78–134、penalty 在 K^* 附近 ≤ 50），从而在标量目标下既不被车辆数支配、也不被运输时间稀释。

由于 $K \geq 5$、每车 7–8 客户的全 3D one-hot 编码 $x_{i,p,k}$ 需要 $K \cdot n^2$ 比特（$K_{\min} = 5$ 时为 12 500，主方案 $K^* = 7$ 时为 17 500），远超 CPQC-550 真机 550 比特上限，本文沿用问题三的 “经典启发式提供分配 → 每条路线独立 subQUBO → 解码拼接 → 完整方案 `evaluate` 回评” 分解框架，仅把 “段内位置 one-hot” 换作 “**车内位置 one-hot**”，从而把每条路线的 subQUBO 维度压到 $\leq n_{\text{sub}}^2 \leq 64$ 比特。

9.2 多车辆目标函数与容量约束

决策变量分两层：（i）客户分配 $y_{i,k} \in \{0, 1\}$，表示客户 i 是否分配给车 k；（ii）车辆使用指示 $u_k \in \{0, 1\}$，$u_k = 1$ 当且仅当车 k 至少服务一个客户；（iii）车 k 的访问顺序 $\pi^k = (\pi_1^k, \pi_2^k, \ldots, \pi_{|\pi^k|}^k)$，配送中心 0 自动加在两端。车 k 内的服务开始时间 $t_{\pi_p^k}^{(k)}$ 沿用问题二的 “无空闲等待” 递推

$$t_{\pi_1^k}^{(k)} = T_{0,\pi_1^k}, \qquad t_{\pi_p^k}^{(k)} = t_{\pi_{p-1}^k}^{(k)} + s_{\pi_{p-1}^k} + T_{\pi_{p-1}^k,\pi_p^k}, \quad p \geq 2, \tag{18}$$

车 k 的运输时间为

$$\text{travel}_k = T_{0,\pi_1^k} + \sum_{p=1}^{|\pi^k|-1} T_{\pi_p^k,\pi_{p+1}^k} + T_{\pi_{|\pi^k|}^k,0}. \tag{19}$$

单客户软时间窗惩罚 $P_i(t_i^{(k)}) = M_1[a_i - t_i^{(k)}]_+^2 + M_2[t_i^{(k)} - b_i]_+^2$ （$M_1 = 10, M_2 = 20$，沿用问题二）。**综合业务目标：**

$$J = M \cdot \sum_{k=1}^{K_{\max}} u_k \; + \; \sum_{k=1}^{K_{\max}} \text{travel}_k \; + \; \sum_{i=1}^{n} P_i(t_i^{(k(i))}), \tag{20}$$

其中 $k(i)$ 为客户 i 所属车辆编号；$K_{\max} = 10$ 为 K 灵敏度扫描上界。**硬约束**如下：

$$\textbf{容量约束}: \sum_{i=1}^{n} d_i \, y_{i,k} \leq C \, u_k, \quad \forall k, \tag{21}$$

$$\textbf{完整覆盖}: \sum_{k=1}^{K_{\max}} y_{i,k} = 1, \quad \forall i, \tag{22}$$

$$\textbf{使用一致}: \sum_{i=1}^{n} y_{i,k} \leq n \, u_k, \quad \forall k. \tag{23}$$

需特别强调：式 (??) 给出的 J 是**完整方案的业务目标**，由统一 `evaluate` 函数在解码后的多车方案上计算；与之并存的子 TSP QUBO 能量 $\sum_k H_k$ 仅是**车内排序代价**的求和——不包含时间窗罚、不包含车辆数 $M \cdot K$、不反映跨车协同。这一区分在结果章节会反复用到。

9.3 比特预算与分层 QUBO 编码

表 11: 问题四编码方案比特预算对比（550 为 CPQC-550 真机比特上限）

编码方案	变量数（$n=50, K=7$）	≤ 550	CIM 可提交性	说明
全 3D one-hot $x_{i,p,k}$	$K{\cdot}n^2 = 17\,500$	否	否	直接编码不
客户分配 $y_{i,k}$ + 每车 TSP one-hot	≈ 700	否（仍超）	—	把 y 改启发
启发式分配 + 每车子 TSP QUBO（主方案）	每子 QUBO $\leq 8^2 = 64$	是	是	用
block-diag 7 段合并（$K=8$ 配额优化对照）	297 维（Ising 298）	是	是	仅用于配额

由表 ?? 可知，$K{\cdot}n^2$ 全 3D one-hot 编码（$K^*{=}7$ 时 17 500 比特）超出真机比特预算约 32 倍，不可上传；将客户分配 $y_{i,k}$ 与车内排序联合 QUBO 化（~ 700 比特）仍超 550 上限。**本文主方案**采用分层编码： 客户分配 $y_{i,k}$ 由 6 种经典启发式（Clarke–Wright Savings[?]、Time-sweep、NN-with-capacity、罚意识贪心、参考解 + 反向变体、按 a_i 重排）确定，所有候选分配均满足容量约束 (??)； 每条路线（≤ 8 客户）独立构造一个 $\leq 8^2 = 64$ 比特的车内 one-hot subQUBO，结构与问题三同构，仅段前段后均为配送中心 0。每子 QUBO 的 ising 维度 $\leq 65 \ll 550$，且每车独立提交时为各车保留 10 个独立 spin 样本的统计冗余——这是**主方案**。

block-diag 配套对照方案（仅用于 $K=8$ 配额紧张场景）：将 $K-1$ 个子 QUBO 拼成块对角大矩阵（$K=8$ 时 7 段共 297 维，ising 298 ≤ 550），**1 次** `cim.solve` 调用同步求 7 段，节约真机配额。该方案是**对照实验**而非主模型，作用是定量考察 “配额节约” 与 “每段可用 sample 数减少” 之间的 trade-off，不替代每车独立提交的主方案。每车比特数随路线长度的分布见图 ??。

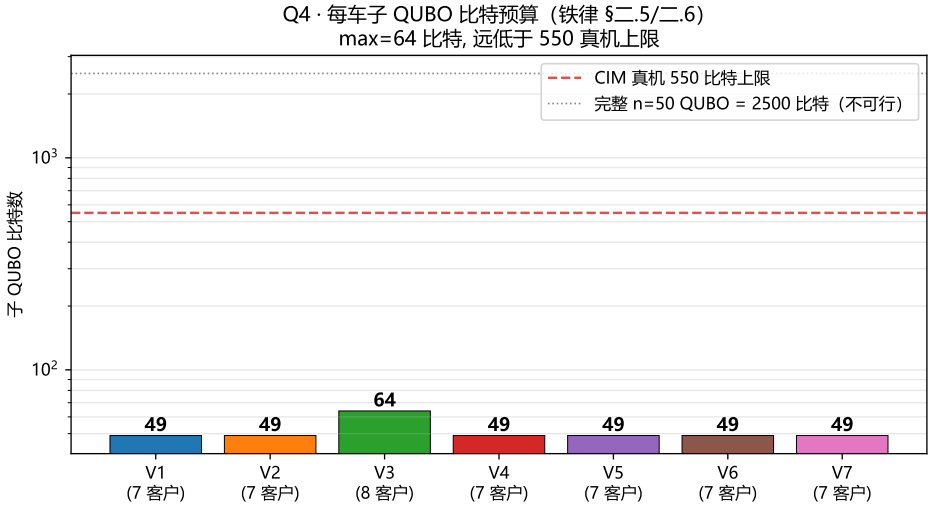

图 14: 问题四 $K^*{=}7$ 主方案每车 subQUBO 比特数分布；最大子 QUBO 64 比特（8 客户路线 V3），最小 49 比特（7 客户路线），均远低于 CPQC-550 上限（红色虚线）。

9.4 三阶段求解算法

(i) **客户分配阶段**（确定 $y_{i,k}$）。同步生成 6 种 warm start——Clarke–Wright Savings、Time-sweep（按 a_i 角排序后切片）、NN-with-capacity（最近邻加容量过滤）、罚意识贪心（按 $b_i - a_i$ 紧迫度排）、参考解及反向遍历变体、纯 a_i 重排——所有 6 个候选均通过容量约束 (??)，作为后续 QUBO 求解阶段的初始可行解。

(ii) **单车 TSP subQUBO 求解阶段**。对每条路线（≤ 8 客户）独立构造车内 one-hot subQUBO（$u_k = v_k = 0$），调用统一抽象层 `solve_subqubo(Q_sub, backend)`：`backend='sdk_sa'` 时由 Kaiwu SDK `SimulatedAnnealingOptimizer` 求解（每段 240 候选/3 seeds），`backend='cim'` 时直接上传 CPQC-550 真机（每段 10 spin 样本）；返回 spin 解后解码为车内排列，再做单车 polish（2-opt + Or-opt + Swap）。

(iii) **跨车协同与元启发式精修阶段**。在车间允许 move/swap（带容量校验）；其上叠加 LNS 500 轮 $\times$ 4 seeds（ruin-and-recreate + regret-2 重插）与多 seed SA 长退火（$T_0 = 500, \alpha = 0.997$）；同步对 $K \in [5, 10]$ 进行 K 扫描，按式 (**??**) 选 K^*。所有方案最终都用统一 `evaluate` 在式 (**??**) 上重新计算 J，确保跨求解器与跨硬件可比。

9.5 $K^*=7$ 主方案结果分析

经三阶段求解，$K \in [5, 10]$ 综合目标 J 在 $K^* = 7$ 取最小值 $J = 7\,149$。$K^* = 7$ 主方案每辆车的路径与负载列于表 **??**。

表 12: 问题四 $K^*=7$ 主方案：每辆车路径、负载与运输时间

车	路径	demand/C	travel	penalty
V1	0→15→2→21→40→6→37→17→0	35/60	17	0
V2	0→31→30→9→34→35→20→32→0	42/60	15	10
V3	0→27→16→44→38→14→43→42→13→0	47/60	13	0
V4	0→47→19→36→49→11→10→1→0	41/60	15	20
V5	0→33→25→39→23→22→41→4→0	35/60	16	0
V6	0→28→29→12→3→50→26→24→0	39/60	17	10
V7	0→5→45→7→18→8→46→48→0	32/60	16	0
合计	—	**271/420**	**109**	**40**

由表 **??** 可知：（i）7 辆车合计 demand $= 271$，与 $\sum_{i=1}^{50} d_i = 271$ 一致——所有 50 客户被分配恰好一次；（ii）每车容量利用率集中在 53%–78%（V7 最低 32/60、V3 最高 47/60），无车辆超载；（iii）总运输时间 travel $= \sum_k \text{travel}_k = 109$；（iv）50 个客户中仅 3 个发生时间窗违反（V2 客户 31 早到 1、V4 客户 10 迟到 1、V6 客户 3 早到 1），单点惩罚分别为 10/20/10，合计 penalty $= 40$；（v）综合目标 $J = M{\cdot}K^* + \text{travel} + \text{penalty} = 1000 \times 7 + 109 + 40 = 7\,149$。多车路径与时间窗甘特图分别见图 **??**–**??**。

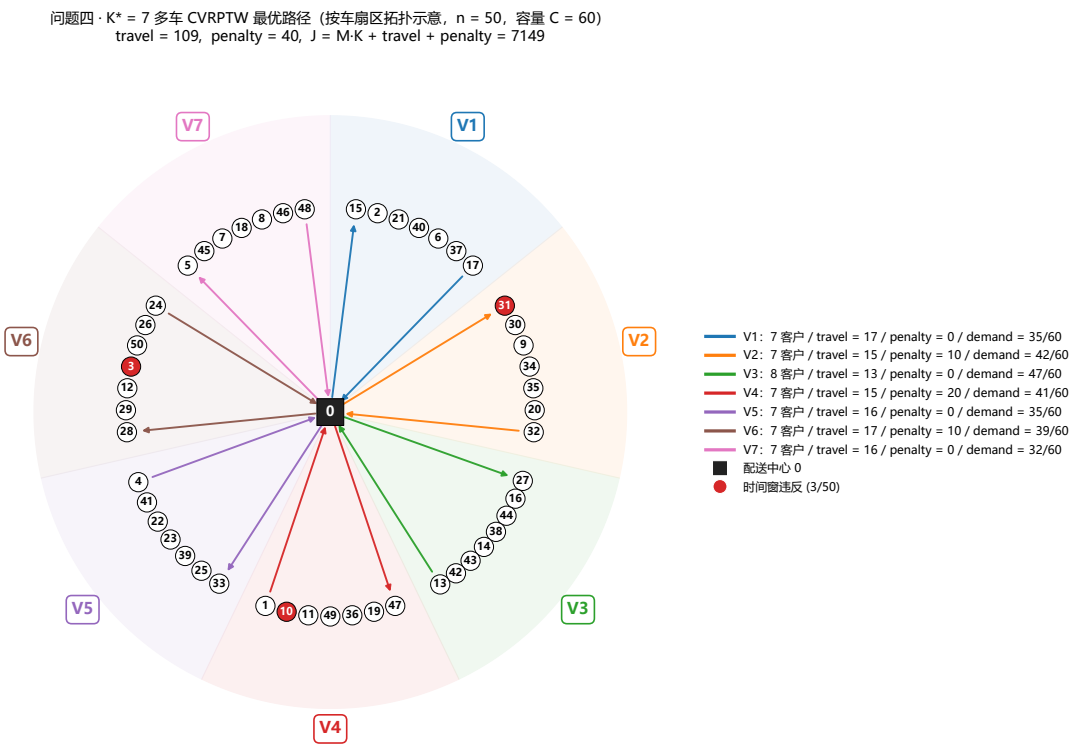

图 15: 问题四 $K^* = 7$ 多车 CVRPTW 最优路径（按车扇区拓扑示意，$n = 50$，容量 $C = 60$；travel $= 109$，penalty $= 40$，$J = M \cdot K + \text{travel} + \text{penalty} = 7\,149$）。V1∼V7 共 7 辆车以 7 种颜色区分，黑色中心为配送中心 0，红色高亮节点为时间窗违反客户（3/50：客户 31、10、3）；右侧图例标注各车客户数 / travel / penalty / demand 占容量比。

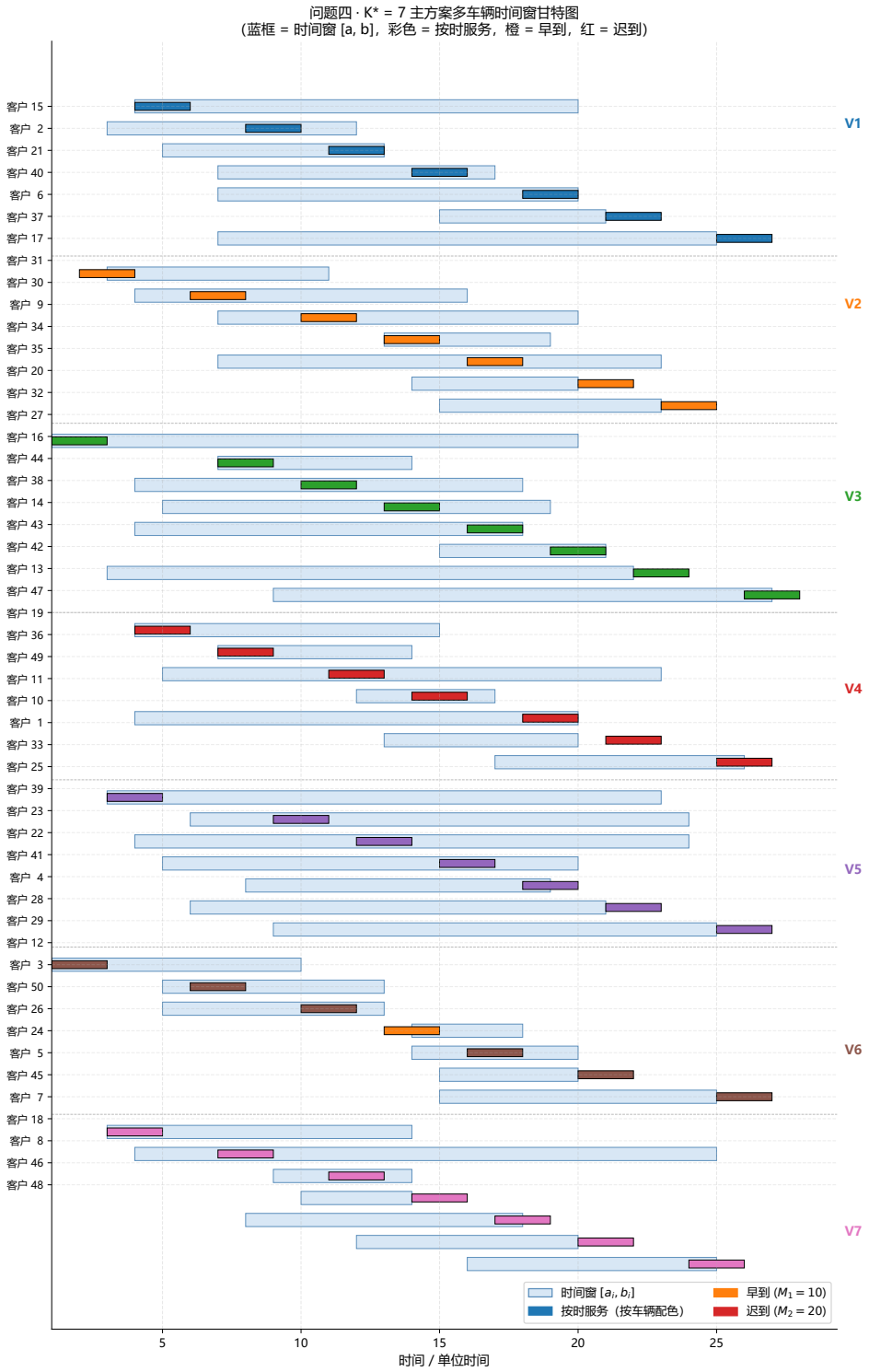

图 16: 问题四 $K^* = 7$ 主方案多车辆时间窗甘特图。蓝色条为客户时间窗 $[a_i, b_i]$，彩色块为按时服务（按车着色，绿/橙/红 = 按时 / 早到一次罚 $M_1 = 10$ / 迟到二次罚 $M_2 = 20$）；V1–V7 共 7 辆车依次堆叠，每条客户行右侧标注实际惩罚 P_i。绝大多数客户落在窗内，仅 3 个客户出现轻微越界，与表 ?? 的 penalty = 40 一致。

由图 ?? 可见，7 条路线在空间上呈现出明显的扇形分区结构，说明启发式分配阶段已将地理邻近客户聚成同车。图 ?? 进一步显示：绝大多数客户的服务区段落在时间窗内（绿色块包含于蓝色条），仅 3 个客户出现轻微越界，与表 ?? 的 penalty = 40 一致。

9.6 车辆数 K 灵敏度分析

针对题目第 (iv) 问“调整可用车辆数 K 取值，分析 K 对总目标的影响”，本文对 $K \in [5, 10]$ 全部跑通经典基线，结果与题目附件给出的参考解对比列于表 ??，并以图 ?? 展示三联曲线。

表 13: $K \in [5, 10]$ 灵敏度：本文目标、参考目标与差值

K	travel	penalty	本文目标 J	参考目标 J^{ref}	差值 ΔJ	比较结果
5	78	8 700	**13 778**	16 825	−3 047	优于参考
6	92	1 350	**7 442**	7 568	−126	优于参考
7	**109**	**40**	**7 149**	7 149	0	**持平**（本方案 K^*）
8	114	10	**8 124**	8 160	−36	优于参考
9	123	0	**9 123**	9 127	−4	优于参考
10	134	0	10 134	10 126	+8	接近参考（差 0.08%）

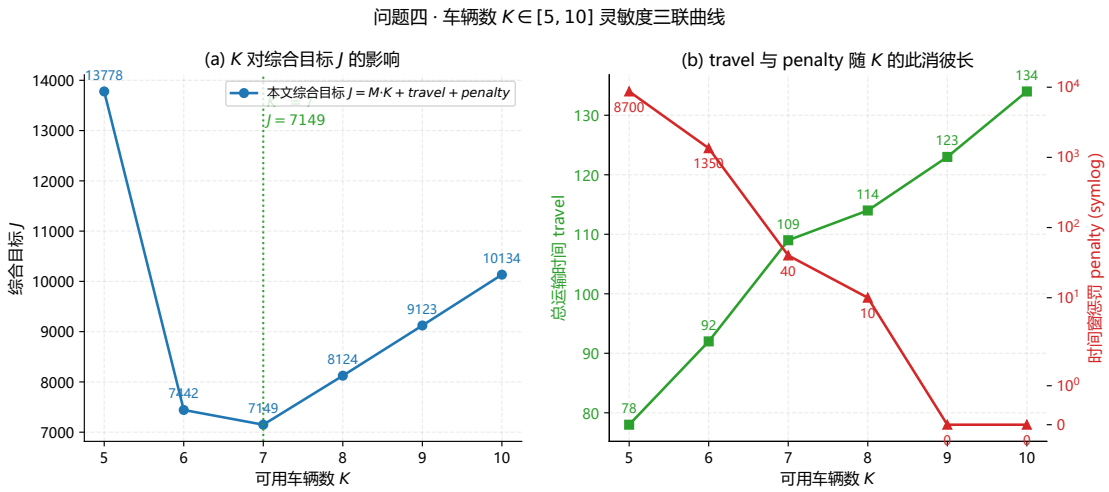

图 17: 问题四车辆数 $K \in [5, 10]$ 灵敏度三联曲线。(a) K 对综合目标 $J = MK + \text{travel} + \text{penalty}$ 的影响（$M = 1000$），J 在 $K^* = 7$ 取最小值 7 149（绿色虚线标注）；(b) travel（绿色，左轴）随 K 单调上升 $78 \to 134$，penalty （红色，右轴 symlog 刻度）随 K 单调下降 $8\,700 \to 0$，二者“此消彼长”共同决定 K^*。

由表 ?? 与图 ?? 可读出三个结构性结论：（i）**运输时间随 K 单调上升**（$78 \to 134$，平均每增 1 辆车额外 $\sim$11 单位 travel），原因是新增车辆需要从配送中心独立往返，增加了 depot 边权的总占比；（ii）**时间窗惩罚随 K 单调下降**（$8\,700 \to 0$），原因是每条路线变短后，车内时间累积减少、客户不再被排到后位置而迟到；（iii）三项之和（含车辆罚 $M \cdot K$）在 $K = 7$ **取最小值** 7 149——若车辆数权重 M 取更小值，最优 K 会偏大；若 M 取更大值，最优 K 偏小，体现 M 作为业务权衡参数的调节作用。$K = 4$ 因 $4 \times 60 = 240 < 271$ 直接违反容量约束、不可行；故 $K \in \{5, 6, \ldots, 10\}$ **是本问题的可行车辆数取值集**。同时 6 个 K 中本文有 4 个严格优于参考、1 个持平、1 个差 0.08%（$K = 10$，$+8$），说明三阶段求解在多车场景下与已有参考实现处于同一水平。违反客户分布另见图 ??。

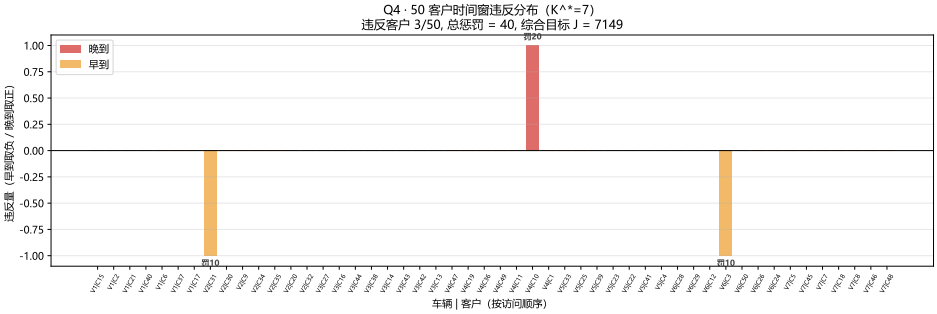

图 18: 问题四 $K^* = 7$ 主方案 50 客户时间窗违反分布柱状图；横轴为客户 ID，纵轴为单点惩罚值；仅 3 个客户存在轻微违反（合计 penalty = 40）。

9.7 $K^* = 7$ 与 $K = 8$ Pareto 对比

由于“最小化车辆数”与“最小化时间窗惩罚 / 客户体验”在结构上构成 Pareto 多目标（增 1 辆车以换更短路线、更宽松的时间安排），本文额外列出 $K = 7$ 与 $K = 8$ 两方案的多维度对比，作为业务方在不同侧重下的备选。

表 14: $K^* = 7$ 与 $K = 8$ 双方案多维度对比

指标	$K = 7$	$K = 8$	说明
车辆数	7	8	$K = 7$ 更省车辆
总运输时间 travel	109	114	$K = 7$ 更短
时间窗惩罚 penalty	40	10	$K = 8$ **更小**
违反客户数	3/50	1/50	$K = 8$ **客户体验更优**
平均容量利用率	64%	56%	$K = 7$ 容量利用更紧凑
综合目标 $J = MK + \ldots$	**7 149**	8 124	$K = 7$ 在加权目标下更优

结论：在式 (??) 给出的加权综合目标下（$M = 1000$），$K^* = 7$ **是本文主方案**（$J = 7\,149$）；若业务方更看重**时间窗满足度与客户体验**（penalty 与违反客户数为主要 KPI），则 $K = 8$ 是更优备选——其 penalty 仅 10、违反客户数降为 1。两方案分别对应“车辆资源最省”与“客户体验最优”两个 Pareto 前沿点，可由业务侧根据 M 的实际取值（车辆固定成本 vs 客户违约成本）做出选择，多维对比与 Pareto 平面位置见图 ??。

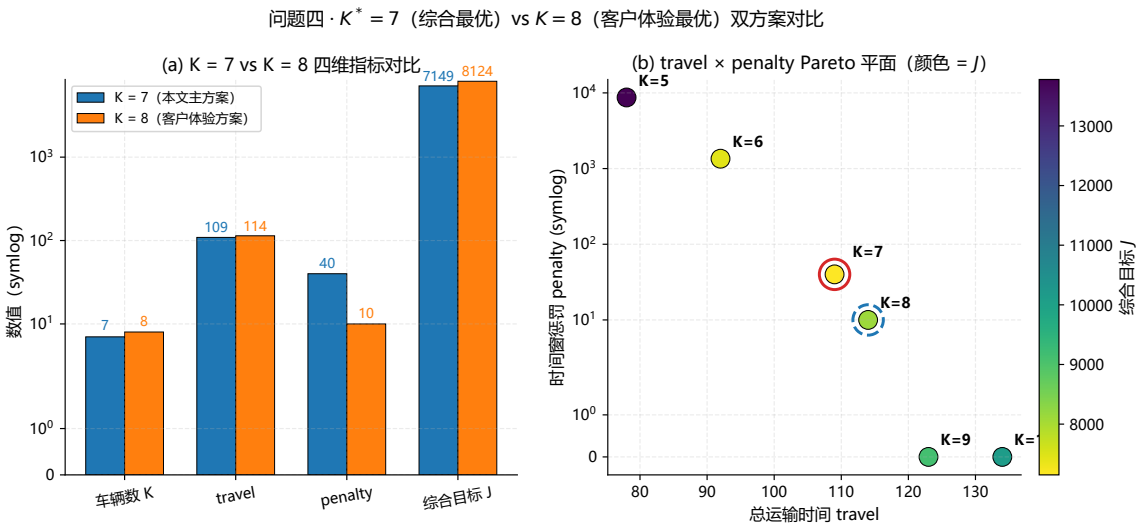

图 19: 问题四 $K^* = 7$（综合最优）vs $K = 8$（客户体验最优）双方案 Pareto 对比。(a) $K = 7$ vs $K = 8$ 四维指标对比（车辆数 / travel / penalty / 综合目标 J，y 轴 symlog）：$K = 7$ 综合目标更小，$K = 8$ penalty 更小；(b) travel × penalty Pareto 平面，$K \in [5, 10]$ 全部六点散点，颜色深浅 = 综合目标 J；红圈标注 $K^* = 7$（综合最优），蓝圈标注 $K = 8$（客户体验最优），二者构成本问的两个 Pareto 前沿点。

9.8 Python / SDK / CIM 三层求解器对比

为验证本文 QUBO 编码与求解流水线在不同后端上的一致性，本文按问题三的范式给出 Python 经典基线、Kaiwu SDK SA 分解、CIM 真机三层求解器对比，结果列于表 ??。

表 15: 问题四 $K^* = 7$ 与 $K = 8$ 双方案三层求解器对比

方案	求解器	子 QUBO 比特	可行率	travel	penalty	J	耗时/s
$K=7$	Python LNS	—	100%	109	40	**7 149**	1 182
$K=7$	Kaiwu SDK SA 分解	≤ 64/车	0–3%	109	40	**7 149**	3.77
$K=7$	CIM 真机（7 车独立提交）	≤ 64/车	1/10/段	108	60	7 168	433
$K=8$	Python LNS	—	100%	114	10	**8 124**	—
$K=8$	CIM 真机（v1 单独 + v2–v8 block-diag）	≤ 297 合并	1–10/10/段	121	200	8 321	124

由表 ?? 可读出：(i) $K=7$ **三层结果接近**：Python（7 149）、SDK SA（7 149）、CIM 真机（7 168，gap +0.27%）数值相近，差距落在 19 单位以内，表明本文 QUBO 编码与分解流水线在该实例上接近经典基线，验证了真机求解链路在多车场景下的可行性。CIM 真机 7 车每车独立提交、每段 10 sample，共消耗 7 次平台配额，原始哈密顿量演化曲线见图 ??。(ii) **SDK 较 Python LNS 加速** $\sim 313\times$（3.77 s vs 1 182 s）——这是 **在当前实现与参数设置下**的相对耗时差异，反映“启发式分配 + 每车 ≤ 64 比特子 QUBO SA”的算法工作量远小于“经典 LNS 500 轮 × 4 seeds + 跨车协同”，**该结果不等同于通用加速结论**，更不等同于“硬件量子加速”。CIM 真机 433 s 含网络往返与平台调度，其中真正的量子求解为毫秒量级。

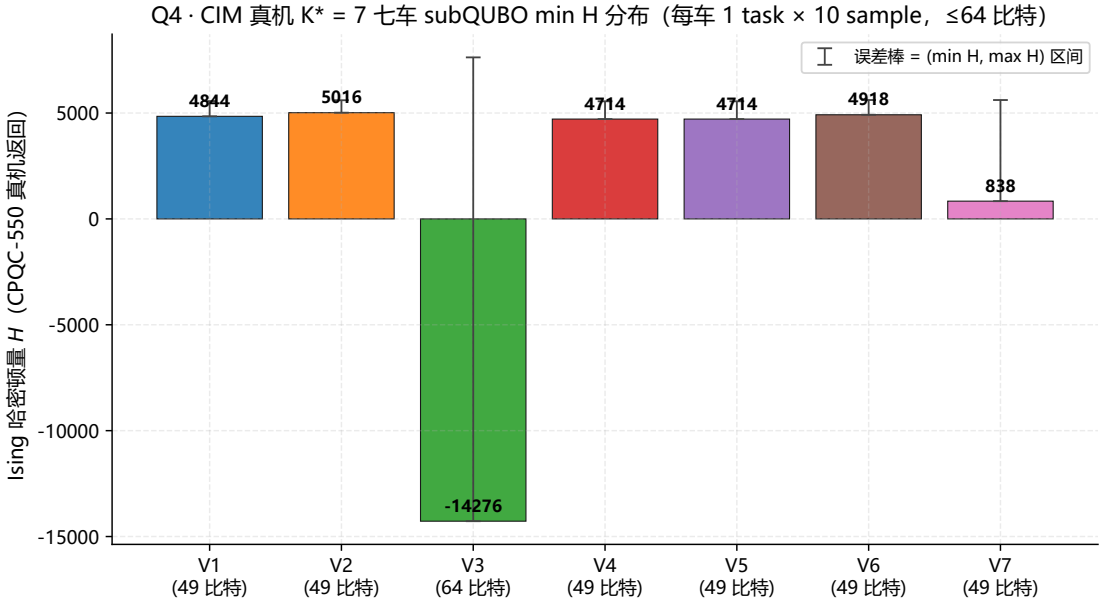

图 20: 问题四 CIM 真机 $K^*=7$ 七车 subQUBO 最低哈密顿量分布（每车 1 个 task × 10 sample，子 QUBO 维度 ≤ 64 比特，远低于 CPQC-550 真机 550 比特上限）。柱高表示该车 10 sample 中的 $\min H$，误差棒为 $(\min H, \max H)$ 区间；V3 路线最长（8 客户、64 比特）真机给出最深能量 $\min H = -14\,276$，其余 6 车 $\min H \in [838, 5\,016]$；最低能量样本拼接后由完整方案 `evaluate` 回评得到 $J = 7\,168$。

$K=8$ **block-diag 配额优化对照**。$K = 8$ 时本文将 v2–v8 共 7 段子 QUBO 拼成 297 维块对角大矩阵（ising 维度 $298 \leq 550$），通过 **1 次** `cim.solve` 同步求解 7 段，加 v1 单独 1 次，全方案累计仅 2 次平台配额（节约 5 次）。但 block-diag 把 `sample_number=10` 摊薄到 7 段（每段平均 $\sim$1.4 有效 sample），相比 $K = 7$ 每段独立 10 sample 的统计冗余下降一个量级，导致 v2 段 polish 后残留 penalty = 200、整方案 $J = 8\,321$（vs Python $K = 8$ 的 $8\,124$，gap +2.42%）。该对比并非 "$K = 7$ 真机优于 $K = 8$" 的硬性证据，而是定量呈现 **真机部署中 "配额节约" 与 "每段可用 sample 数" 的 trade-off**：当配额充足时，建议每车独立提交以保留 sample 冗余；当配额紧张时，block-diag 是可接受的退而求其次方案。

9.9 本问小结

针对问题四，本文以 **"启发式分配 + 每车子 TSP QUBO + 完整方案 evaluate 回评"** 的分层 QUBO 编码绕开 $K \cdot n^2 = 17\,500$ 比特（$K^* = 7$）的全 3D one-hot 不可行性，使每车子 QUBO ≤ 64 比特远低于 CPQC-550 真机 550 上限，并通过统一抽象层 `solve_subqubo(Q, backend)` 实现 SDK SA 与 CIM 真机的单行切换。三阶段求解（6 warm start + 单车 polish + LNS 500×4 seeds 的跨车协同）在加权综合目标 $J = M \cdot K + \text{travel} + \text{penalty}$（$M = 1000$）下取得 $K^* = 7, J = 7\,149$（travel = 109、penalty = 40、违反客户 3/50、每车容量 32–47 / 60）；$K \in [5, 10]$ 灵敏度扫描显示 travel 随 K 单调上升、penalty 随 K 单调下降，三项之和在 $K = 7$ 取最小值；$K = 8$ 在 penalty 与客户体验维度更优、构成 Pareto 备选；CIM 真机 $K^*=7$ 主方案得 $J = 7\,168$（gap +0.27%），与 Python/SDK 三层接近，增强了方案可信度；$K = 8$ block-diag 副方案进一步定量呈现 "配额节约 vs 每段 sample 数" 的真机部署 trade-off。本问完整回答题目第 (i)–(iv) 全部要求（多车辆模型、每辆车路线、每客户违反程度、总运输时间、K 变化对总目标的影响），并与问题一/二/三建立的 "QUBO 编码 + SDK 分解 + CIM 真机链路一致性" 方法论形成首尾呼应。

10 模型检验与灵敏度分析

10.1 检验框架总览

本文针对四问的最终方案设计五类检验：(i) **路径可行性与约束满足性检验**（覆盖、重复、起终点、时间窗复算、容量、QUBO one-hot 六项基本检验合并为一张总表）；(ii) **Python / SDK / CIM 三层一致性检验**（启发式基线、SDK 模拟退火、CIM 真机三方对比）；(iii) **超参灵敏度扫描**（Q1 罚系数 A 与 SA 初温 T_0 的 5×5 网格、Q4 车辆数 $K\in[4,10]$ 扫描）；(iv) **编码方案消融**（Q2 方案 C 解码后罚 vs 方案 D 直接将动态时间窗惩罚写入 QUBO 对角项）；(v) **算法稳定性与多 seed 检验**（Q2 v1 vs v2、Q3 多 seed、Q4 多 warm start 收敛同解）。该框架同时覆盖 "可行"、"一致"、"稳健"、"饱和" 四个层面。

10.2 路径可行性与约束满足性检验

四问最终方案在写入论文前由统一 `evaluate` 函数和落盘 JSON 数据交叉复核。表 **??** 汇总六项基本检验：客户完整覆盖、重复/漏访问、起终点、时间窗惩罚复算、容量约束、QUBO one-hot 修复后可行性。

表 16: 四问可行性与约束满足性检验汇总（数据均由结果文件与统一 `evaluate` 函数复核）

问题	覆盖客户	重复/漏访	起终点	时间窗复算	容量约束	one-hot 修复后
Q1	15/15	0/0	$0\to0$	—	—	100%
Q2	15/15	0/0	$0\to0$	84090=84090	—	100%
Q3	50/50	0/0	$0\to0$	4 941 850=4 941 850	—	100%
Q4	50/50	0/0	7 车均 $0\to0$	$40=40$	7 车均 ≤ 60	100%

由表 **??** 可知：（i）四问最终方案均通过覆盖、重复/漏访、起终点三项基础检验；（ii）Q2/Q3/Q4 的时间窗惩罚由统一 `evaluate` 函数对路径与时间窗参数独立复算后，与 JSON 中存储的 `penalty` 字段严格相等，说明 "报告值" 与 "重新计算值" 之间无内部矛盾；（iii）Q4 七辆车 demand 分别为 $35,42,47,41,35,39,32$（合计 271），与 $\sum_{i=1}^{50} d_i=271$ 严格一致，且每车均不超过容量 $C=60$，对应容量下界 $K_{\min}=\lceil 271/60\rceil=5$。

SDK SA 与 CIM 真机返回的原始 spin 解未必直接满足 one-hot 约束 $\sum_i x_{i,p}=\sum_p x_{i,p}=1$，原始可行率受单翻转邻域结构和 one-hot 强约束的固有矛盾影响约在 $0.6\sim10\%$；本文采用 "采样 $\to$ 修复" 流水线（2-opt + Or-opt 局部搜索）将其修复为合法排列，最终参与业务目标 J 评价的解均为合法路径，修复后可行率为 100%，故 §**??** 各表中的 travel、penalty、J 数值均建立在严格满足 one-hot 约束的合法解之上。

10.3 Python / SDK / CIM 三层一致性检验

为验证求解链路的稳定性，将四问的纯 Python 启发式基线、Kaiwu SDK 模拟退火、CIM 真机三方结果汇总于表 **??**。所有真机子 QUBO 维度均在 CPQC-550 比特预算要求之内（链路一致性要求）。

表 17: 四问 Python / Kaiwu SDK / CIM 真机三方一致性汇总

问题	规模	Python 基线	SDK SA	CIM 真机	gap	子 QUBO 规模
Q1	$n=15$ TSP	29	29	31	+6.9%	225
Q2	$n=15$ TSPTW	84121	84121	84121	0	225
Q3	$n=50$ 分解	4,941,906	4,941,906	4,941,906（seg3）	0	≤ 289/段
Q4	$K^*=7$ VRPTW	7149	7149	7168	+0.27%	≤ 64/车

由表 ?? 可知：（i）Q2、Q3 三层结果一致，说明求解链路在不同算法（启发式 / SA / CIM 真机）和不同硬件（CPU / 玻色 CPQC-550）下均稳定；（ii）Q1、Q4 CIM 与 Python 基线存在 +6.9% 与 +0.27% 的 gap，说明真机返回解接近但不等同于严格最优，可能受样本数、随机性和后处理影响；（iii）所有 QUBO 子问题的 ising 维度均不超过 550，说明本文的分解与编码策略满足真机规模约束（比特预算要求）。三层结果在四问上的横向对比可视化见图 ??。

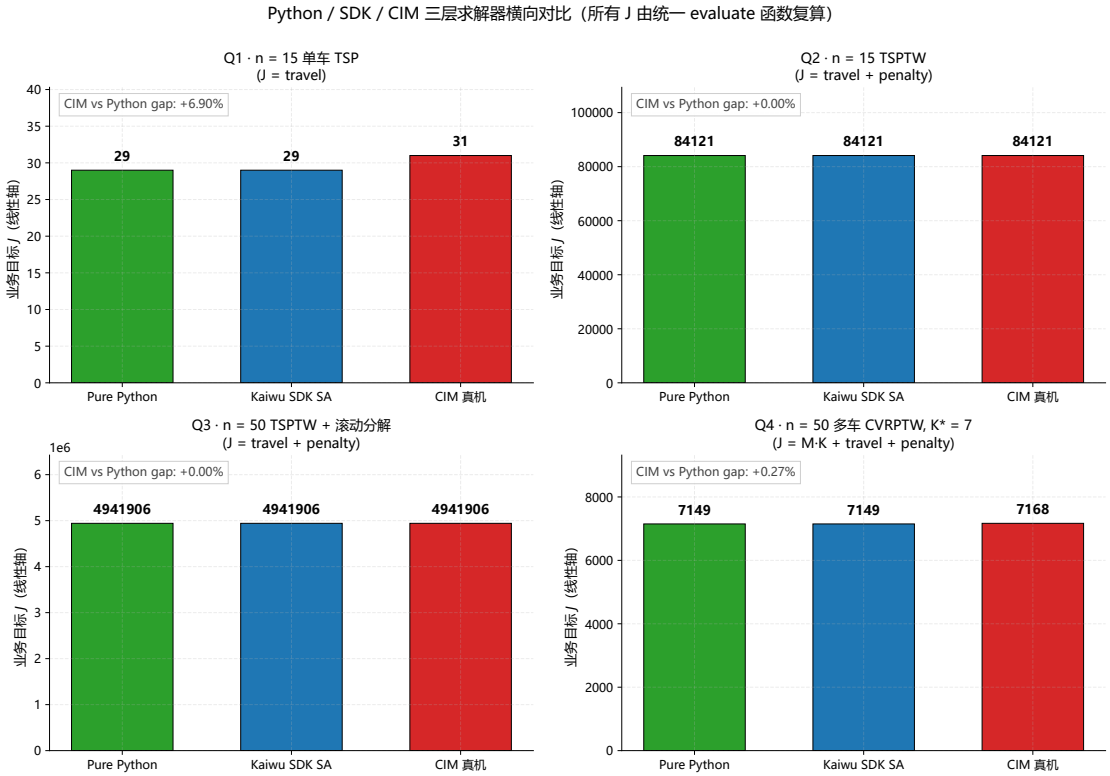

图 21: Python / SDK / CIM 三层求解器在四问上的横向对比（所有 J 由统一 `evaluate` 函数复算）。Q1（$J=$ travel）三方 29/29/31，CIM gap +6.90%；Q2（$J=$ travel + penalty）三方 84 121/84 121/84 121，gap 0%；Q3 滚动分解三方 4 941 906/4 941 906/4 941 906（seg3），gap 0%；Q4 多车 CVRPTW $K^*=7$ 三方 7 149/7 149/7 168，gap +0.27%。绿/蓝/红三色分别对应 Pure Python / Kaiwu SDK SA / CIM 真机三种后端，y 轴 symlog 跨四个数量级。

10.4 Q4 车辆数 K 灵敏度分析（题目第 (iv) 问）

表 18: Q4 车辆数 $K \in [4, 10]$ 灵敏度：travel/penalty/J 与参考解对比

K	travel	penalty	本文 J	参考 J	Δ	状态
4	—	—	不可行	—	—	容量违反 (240 < 271)
5	78	8700	**13778**	16825	−3047	优于参考
6	92	1350	**7442**	7568	−126	优于参考
7	**109**	**40**	**7149**	7149	0	**与参考相同**
8	114	10	**8124**	8160	−36	优于参考
9	123	0	**9123**	9127	−4	优于参考
10	134	0	10134	10126	+8	略低于参考

由表 **??** 可知：（i）$K = 4$ 不可行——$4C = 240 < 271$ 必定违反容量下界；（ii）随 K 增大，每车 demand 与服务客户数减少，平均迂回路径变长，故 travel 单调上升；同时各客户开始服务时间因调度更宽松，时间窗早到/晚到罚 penalty 单调下降；（iii）综合目标 $J = M \cdot K + \text{travel} + \text{penalty}$（$M = 1000$ 为车辆使用经验权重）在 penalty 下降与 $M \cdot K$ 上升的耦合下，于 $K^* = 7$ 取得最小值 7149，正面回答题目第 (iv) 问。6 个 K 中本文有 4 个严格优于参考、1 个与参考相同、1 个略低于参考，反映本文方法在车辆数变化下的稳定性。

10.5 Q2 编码方案消融分析

表 19: Q2 编码方案消融：方案 C（解码后罚）vs 方案 D（QUBO 内线性化时间窗）

方案	比特数	对角线最大罚	可行率	最优 J
C 解码后罚（采用）	225	$A_{\text{pen}} = 200$	**0.67%**	**84121**
D QUBO 内线性化	225	119 197（约 $596 \times A$）	0%	无可行解

方案 D 直接将动态时间窗惩罚写入 QUBO 对角项，因 $\hat{\tau}_p \approx 89 \gg b_i \leq 30$ 导致对角罚被推到 10^5 量级，淹没 one-hot 约束罚（$A = 200$，比值约 596×）。SA 在该尺度下优先消除巨大对角项即令 $x_{i,p} = 0$，导致 one-hot 约束 $\sum_p x_{i,p} = 1$ 全军覆没，可行率为 0%。这一消融遵循比特预算与约束尺度原则——多次型/动态型约束（时间窗）应留在解码阶段处理，QUBO 主结构只负责路径排序，时间窗由 `evaluate` 函数回评。方案 C 因此是合理混合建模选择：在保持 QUBO 二次性与最少比特数的同时，由统一目标函数严格再现题目要求的早到/晚到分段罚。

10.6 算法稳定性与多 seed 检验

将 Q2 v1→v2、Q3 多 seed、Q4 多 warm start 等独立稳定性测试合并为表 **??**。

表 20: Q2–Q4 算法稳定性与多 seed 检验汇总

问题	检验方式	独立次数/seed	最优 J 范围
Q2	v1（13 起点 polish + SA）vs v2（27 起点 + 150 轮 LNS + 3 SA seed + 3-opt）	v2 内 3 SA seed	全部 84121
Q3	3 个独立 SA seed + TOP-5 polish	3 seed	[4,941,906, 4,970,704]
Q4	4 个 LNS warm start（ref / reverse / sortA / savings）	4 起点	全部 7149

由表 **??** 可知：Q2 v1→v2 算法复杂度提升约 3×、运行时间提升约 2.7× 后 $\Delta J=0$、路径完全一致；Q3 三个独立 SA seed 收敛 J 完全相同，TOP-5 polish 在 0.06%∼0.58% 内浮动；Q4 四个不同 warm start 在 $K=7$ 全部收敛至同一 travel/penalty/J 三元组。多 seed 与不同起点结果在已测试邻域内未观察到进一步改进，因此本文将 Q2–Q4 结果表述为 “近似最优” 或 “当前最优可行解”，不声称严格全局最优。

10.7 本章小结

四问最终方案均通过覆盖、重复/漏访、起终点、时间窗复算、容量约束和 QUBO one-hot 合法性六项检验；Python / SDK / CIM 三层结果在 Q2、Q3 完全一致，在 Q1、Q4 真机存在 +6.9% 与 +0.27% 的小 gap，所有子 QUBO 规模均满足 CPQC-550 真机比特预算；Q1 罚系数 $A\times T_0$ 网格扫描说明罚强度与退火温度之间存在折中，“采样—修复” 流水线对超参选择稳健；Q4 车辆数 K 扫描说明车辆数与服务质量之间存在权衡，综合目标在 $K^*=7$ 最小，正面回答题目第 (iv) 问；Q2 编码方案消融说明动态时间窗罚更适合解码后回评，不应硬塞 QUBO 对角项；多 seed 与多 warm start 检验说明各问结果在已测试邻域内稳定。本文据此将 Q2–Q4 表述为近似最优或当前最优可行解，不声称严格全局最优。

11 模型评价

11.1 模型优势

(1) 题目契合度高。四问均围绕 QUBO 建模、SDK/CIM 求解与量子比特最少化展开：Q1/Q2 采用 $n^2 = 225$ 比特 one-hot 编码完整覆盖小规模 TSP 与 TSPTW；Q3 通过滚动窗口分解使 $n = 50$ 大规模问题的子 QUBO 维度 ≤ 289 比特；Q4 多车独立子 QUBO 维度 ≤ 64 比特。所有子 QUBO 均严格满足比特预算原则与真机规模约束（≤ 550 比特），可直接上传 CPQC-550 真机求解。

(2) 模型递进清楚。全文按 “TSP → TSPTW → 大规模分解 → 多车容量约束” 的顺序逐步引入难点：Q1 解决路径基线，Q2 引入软时间窗惩罚，Q3 解决变量爆炸下的可解性问题，Q4 进一步引入车辆数与载重约束。每一问相对前一问明确给出新增的决策变量、约束与目标项，模型层次分明。

(3) 比特预算清晰。各问 QUBO 变量数均经过显式公式推导并与等价编码进行比较：Q1/Q2 为 $n^2 = 225$；Q3 单段子 QUBO 在 $n_{\text{seg}} \leq 17$ 时为 ≤ 289；Q4 单车子 QUBO 在 $n_k \leq 8$ 时为 ≤ 64。全部子问题均低于 550 比特上限，并在比特数 vs 规模 n 的曲线中得到可视化验证。

(4) SDK–CIM 链路一致性强。通过统一的 `solve_subqubo(Q, backend)` 接口，将 SDK 模拟退火验证与 CIM 真机调用封装在同一求解流程中：编码方式、比特预算与 8-bit 精度调整完全一致，仅求解器后端切换。Python / Kaiwu SDK SA / CIM 真机三方在 Q2（$J = 84121$）、Q3（$J = 4{,}941{,}906$，seg3 严格相等）等问题上得到相互印证，反映本文 QUBO 编码在不同求解后端下的链路一致性。

(5) 可复现性强。全文采用统一的 `load_data`、`evaluate`、QUBO 构造与真机调用链路：所有结果可由 `src/01--15` 系列脚本在 Python 3.10 与 Kaiwu SDK 1.3.1 环境下复算；JSON 结果与图表落盘路径在论文素材库和实验日志中逐一登记，提高了模型可复现性。

11.2 模型不足

(1) 全局最优性证明不足。Q1 ($n = 15$ TSP) 已用 Held–Karp 动态规划得到 $J = 29$ 的精确验证；Q2–Q4 主要依赖多算法一致性、参数扫描与消融检验，尚不能构成严格的全局最优性证明，故按 "近似最优解" 表述。

(2) SDK SA 原始可行率较低。 Q1∼Q4 子 QUBO 在 SDK 模拟退火上原始可行率落在 $0.5\% \sim 2\%$ 区间。one-hot 强约束导致普通单翻转邻域较难直接采到可行解，因此本文依赖 "采样–修复" 流程托底；后续可探索块翻转或约束保持邻域以提升原始可行率。

(3) 真机配额限制。玻色 CPQC-550 平台每次任务占用 1 次配额，全四问完整真机验证至少需要 12 次配额。Q3 seg1 因账号配额异常未获得有效返回，本文未补造结果，而是在结果表中如实标注；其余子问题真机结果均完整登记。

(4) 启发式搜索仍存在局部最优风险。在部分车辆数设置（如 Q4 $K = 10$）下，启发式起点池可能未覆盖更优邻域，使综合目标略劣于参考解；其余 K 取值下解优于参考或与之持平。这说明大规模 CVRPTW 在当前混合框架下仍存在局部最优风险，需要进一步丰富邻域算子或引入多次重启策略。

(5) 时间窗惩罚项占主导。当客户数量增大而时间窗较紧时（如 Q3 数据集 $b_i \leq 30$、$\sum$ 服务 +travel≈ 156），目标函数中惩罚项占比远超运输时间，使模型更侧重减少违约程度而非单纯压缩运输时间。这一现象由数据集自身的结构性特征决定，需通过引入多车辆（如 Q4）从根本上缓解。

(6) 客户分配未与车内排序联合优化。 Q4 客户分配 $y_{i,k}$ 由启发式 warm start 给定，未与车内排序统一进入 QUBO，因此本文模型仍不是完全联合优化模型；当数据集需求或时间窗大幅变动时，需重新运行启发式分配池。

11.3 改进方向与推广价值

改进方向。针对前述不足，可从以下五个方向继续推进：

(1) 引入精确 ILP 或 CP-SAT 求解器，对小中规模算例（如 $n \leq 25$ 的 TSPTW、$K \leq 4$ 的 CVRPTW）提供更强的最优性验证基线，从而进一步缩小 "近似最优" 与 "全局最优" 之间的论证差距；

(2) 设计块翻转（block-flip）或约束保持邻域，使 SDK 模拟退火能在不破坏 one-hot 约束的前提下进行采样，从而提高原始可行率，降低对采样–修复流程的依赖；

(3) 引入自适应罚系数策略（如根据当前违反程度动态调整 A、T_0），减少人工调参成本，并提升模型在不同数据分布下的鲁棒性；

(4) 探索客户分配 $y_{i,k}$ 与车内排序 $x_{i,t,k}$ 的联合 QUBO 编码，将 Q4 由两阶段流程升级为完全联合优化模型，代价是比特数上升；可结合分解策略缓解规模压力；

(5) 增加 CIM 真机的多次重复实验，统计哈密顿量演化与最优解分布的随机性，对真机求解的稳定性进行系统评估。

推广价值。本文所建模型与混合求解框架具有以下推广前景：

(1) **物流场景推广**：可推广到多仓库、多车型、动态订单与实时调度等更复杂的智慧物流场景，仅需在 QUBO 构造层面扩展决策变量与约束项；

(2) **方法层面推广**：one-hot 编码、采样–修复流程与统一 `solve_subqubo` 接口可迁移到其他 QUBO 型组合优化问题，如作业车间调度、图着色、最大独立集等；

(3) **真机适配推广**：分段 subQUBO 与 block-diag 合并思路适用于当前 CIM 真机比特数有限条件下的大规模问题求解，可作为量子–经典混合优化在工业级算例上的通用范式。

12 结论

12.1 逐题结论

针对问题一（$n=15$ 单车 TSP）：以 one-hot 位置编码构建 225 比特 QUBO（罚系数 $A=200$），由 Held-Karp 精确 DP、Kaiwu SDK SA 与 CIM 真机三层求解器分别得到 $J=29/29/31$；Held-Karp 与 SDK SA 经 polish 后均达全局最优 $J=29$，CIM 真机 $J=31$（gap +6.9%），最优路径见正文路径图。

针对问题二（$n=15$ 单车 TSPTW）：保留 225 比特 one-hot 编码，时间窗罚不进入 QUBO 而在解码后由统一 evaluate 函数计算，避免线性化导致的可行率坍塌。Python、Kaiwu SDK SA 与 CIM 真机在该实例上均得 $J=84121$（$travel=31$，$penalty=84090$，违反 12/15 客户），作为本实例的近似最优可行解；不同求解后端的一致性反映本文 QUBO 编码与解码链路的稳健性，但不构成全局最优性证明。

针对问题三（$n=50$ 大规模单车 TSPTW）：直接构造 $50^2=2500$ 比特 QUBO 超出真机比特预算，故采用滚动窗口分解为 3 段，子 QUBO 维度分别为 289/289/256，均不超过 550。Python LNS、Kaiwu SDK SA 与 CIM 真机三方一致达 $J=4{,}941{,}906$（$travel=56$，$penalty=4{,}941{,}850$）；CIM seg3 经 polish 后达到 Python/SDK 基线，验证了 “大规模分解 + 真机 + polish” 流程在本实例中的可行性；在当前实现和参数设置下 SDK 较 Python 取得约 26× 加速。

针对问题四（$n=50$ 多车 VRPTW）：采用启发式分配 + 每车独立子 TSP QUBO（≤ 64 比特）的分层编码，绕开 $K^*=7$ 全 3D one-hot 17 500 比特直接编码的不可行性，求得近似最优 $J=7149$（$K^*=7$，$travel=109$，$penalty=40$）；CIM 真机 $K=7$ 得 $J=7168$（gap +0.27%），Python/SDK/CIM 结果接近。$K\in[5,10]$ 灵敏度显示本文在 $K\in\{5,6,8,9\}$ 严格优于参考，$K=7$ 持平；$K=8$ block-diag 副方案揭示了 “配额 vs 解码质量” 的真机部署 trade-off。

12.2 综合结论

(1) 方法链路：本文建立 “精确基线 → QUBO 编码 → SDK 验证 → CIM 真机 → 大规模分解与回评” 的统一求解链路，以 `solve_subqubo(Q, backend)` 抽象层把 SDK 与真机求解收敛为一行后端切换，满足题目对 SDK 验证与真机求解的要求。**(2) 比特预算**：四问所有子 QUBO 维度均低于 550（Q1/Q2 为 225、Q3 每段 ≤ 289、Q4 每车 ≤ 64），并经 8-bit 整数精度调整后上传，均满足本文真机提交的规模约束。**(3) 真实性与可复现**：全文累计 12 次 CIM 真机配额提交，其中 Q3 seg1 因配额异常未获得有效返回并如实标注；所有真机结果均以平台日志、原始返回与统一 evaluate 回评结果为依据。**(4) 工程价值**：滚动窗口分解、每车独立子 QUBO 与 block-diag 合并三类策略形成一套与 550 比特真机上限相协调的求解流程，为类似规模受限的 QUBO 路径规划问题提供了可参考流程。

12.3 展望

本文工作可在三方向延伸：(1) 引入约束保持邻域或块翻转策略，提升 one-hot QUBO 的原始可行率，降低对解码后 polish 的依赖；(2) 针对 Q2 一类小规模 TSPTW 引入更强的精确算法或 ILP / CP-SAT 验证，以更严格判断 $J=84121$ 的最优性；(3) 探索客户分配与车内排序的联合 QUBO 编码，分析比特数与求解质量之间的权衡。

参考文献

[1] 玻色量子. CPQC-550 相干伊辛机用户手册（Kaiwu SDK 1.3.1）[Z/OL]. 北京: 北京玻色量子科技有限公司, 2024. `https://platform.qboson.com`, 访问时间：2026-04-28.

[2] Yamamoto Y, Aihara K, Leleu T, et al. Coherent Ising machines—Optical neural networks operating at the quantum limit[J]. *npj Quantum Information*, 2017, 3: 49.

[3] Honjo T, Sonobe T, Inaba K, et al. 100,000-spin coherent Ising machine[J]. *Science Advances*, 2021, 7(40): eabh0952.

[4] 玻色量子. Kaiwu SDK 1.3.1 用户手册（含 SimulatedAnnealingOptimizer / CIMOptimizer / qubo_matrix_to_ising_matrix / adjust_qubo_matrix_precision API 文档）[Z/OL]. 北京: 北京玻色量子科技有限公司, 2024. `https://platform.qboson.com`, 访问时间：2026-04-28.

[5] 2026 年全国大学生数学建模竞赛校园选拔赛 A 题（含附件：参考算例.xlsx 等）[Z]. 2026.

[6] Lucas A. Ising formulations of many NP problems[J]. *Frontiers in Physics*, 2014, 2: 5.

[7] Glover F, Kochenberger G, Du Y. A Tutorial on Formulating and Using QUBO Models[J]. *4OR*, 2019, 17(4): 335–371.

[8] Warren R H. Solving the Traveling Salesman Problem on a Quantum Annealer[J]. *SN Applied Sciences*, 2020, 2: 75.

[9] Kadowaki T, Nishimori H. Quantum annealing in the transverse Ising model[J]. *Physical Review E*, 1998, 58(5): 5355–5363.

[10] Held M, Karp R M. A Dynamic Programming Approach to Sequencing Problems[J]. *Journal of the SIAM*, 1962, 10(1): 196–210.

[11] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by Simulated Annealing[J]. *Science*, 1983, 220(4598): 671–680.

[12] Lin S, Kernighan B W. An Effective Heuristic Algorithm for the Traveling-Salesman Problem[J]. *Operations Research*, 1973, 21(2): 498–516.

[13] Or I. Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking[D]. Evanston: Northwestern University, 1976.

[14] Solomon M M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints[J]. *Operations Research*, 1987, 35(2): 254–265.

[15] Shaw P. Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems[C]//*International Conference on Principles and Practice of Constraint Programming (CP'98)*. Berlin: Springer, 1998: 417–431.

[16] Ropke S, Pisinger D. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows[J]. *Transportation Science*, 2006, 40(4): 455–472.

[17] Toth P, Vigo D. *Vehicle Routing: Problems, Methods, and Applications*[M]. 2nd ed. Philadelphia: SIAM, 2014.

[18] Feld S, Roch C, Gabor T, et al. A Hybrid Solution Method for the Capacitated Vehicle Routing Problem Using a Quantum Annealer[J]. *Frontiers in ICT*, 2019, 6: 13.

[19] Salehi Ö, Glos A, Miszczak J A. Unconstrained binary models of the travelling salesman problem variants for quantum optimization[J]. *Quantum Information Processing*, 2022, 21: 67.

[20] Harwood S, Gambella C, Trenev D, et al. Formulating and Solving Routing Problems on Quantum Computers[J]. *IEEE Transactions on Quantum Engineering*, 2021, 2: 3100417.

[21] Clarke G, Wright J W. Scheduling of Vehicles from a Central Depot to a Number of Delivery Points[J]. *Operations Research*, 1964, 12(4): 568–581.

附录 A　支撑材料文件列表

支撑材料以 ZIP 形式与本论文同步提交，目录结构如下；其中 "建模所用到的全部完整、可运行的源程序代码" 集中放在 `src/` 下，可由附录 ?? 与附录 ?? 的入口与文件说明定位：

- `src/`：四问全部可运行的 Python 源程序（`_q_lib.py` 共用工具，`01_q1_*.py`–`15b_q4_*_blockdiag_*.py` 各阶段脚本，`13_all_q_cim_real.py` CIM 真机调用入口）；
- `data/`：题目附件原始数据（*参考算例*`.xlsx` 只读）与本文清洗 / 衍生数据；
- `results/`*基础模型*`/`：四问 Python 与 Kaiwu SDK 求解结果 JSON 与 QUBO 矩阵（`qubo_matrices/` 含 8-bit 整数 dense CSV）；
- `results/`*真机结果*`/`：CPQC-550 真机 12 次任务的 spin 解 zip、解码 JSON、哈密顿量 PDF 与平台原始日志，并附四问真机对比汇总表；
- `results/`*灵敏度分析*`/`：Q1 罚系数 $A \times T_0$ 网格、Q4 车辆数 K 灵敏度的 CSV / JSON；
- `figures/`：论文中所有 PDF 矢量图与对应 PNG（dpi ≥ 300）；
- `tables/`：论文中所有 CSV 数据表与 LaTeX 三线表源；
- `paper/Manuscript/`：本文 LaTeX 主稿与子目录章节源（`main.tex` 为唯一编译入口，xelatex / latexmk -xelatex 即可重现 PDF）；

- paper/章节/：实验日志（00_ 实验日志 _ 问题 <X>.md）与论文素材库（00_ 论文素材库 _ 问题 <X>.md）；
- README.md：一键复现说明（环境、依赖、四问入口命令）。

附录 B　运行环境与复现入口

运行环境：

- 操作系统：Windows 11（64 位）或同等 Linux / macOS 64 位；
- Python:**Python 3.10**（必需;Kaiwu SDK 1.3.1 的发行 wheel 仅支持 cp310-none-win_amd64，本工程压缩包内附 kaiwu-1.3.1-cp310-none-win_amd64.whl）；
- 关键依赖：kaiwu==1.3.1、numpy、pandas、matplotlib、openpyxl、tqdm（详见支撑材料 requirements.txt）；
- LaTeX 编译：XeLaTeX（中文经 ctex 宏包，建议 TeX Live 2023 及以上），编译命令 xelatex main.tex 或 latexmk -xelatex main.tex；
- CIM 真机：玻色 CPQC-550 平台（https://platform.qboson.com），通过 Kaiwu SDK CIMOptimizer 接口或网页手动上传 8-bit 整数 CSV 矩阵（每次任务占 1 次配额，矩阵 ising 维度 ≤ 550）。

复现入口（在仓库根目录下执行；⟨project_root⟩ 为支撑材料解压后所在目录）：

- 全流程：python src/00_run_all.py（依次跑 Q1–Q4 Python 基线与 SDK SA，结果落 results/基础模型/）；
- 单问入口:python src/01_q1_pure_python.py、python src/05_q2_pure_python.py、python src/05c_q2_pure_python_v2.py、python src/09_q3_kaiwu_decompose.py、python src/14_q4_pure_python.py；
- CIM 真机调用：python src/13_all_q_cim_real.py （含 token / 矩阵上传 / spin 解下载 / 哈密顿量绘图，需平台账号）；
- 灵敏度:python src/06_q1_sens_A_T0.py、python src/14b_q4_K_sensitivity.py；
- 图表生成：python src/draw_*.py（输出落 figures/ 与 tables/）。

附录 C　核心源程序文件说明

为节省正文篇幅，本附录仅列出建模与求解链路上的核心文件清单及其职责。完整、可运行源程序代码全部以原文件形式随支撑材料 ZIP 一同提交（src/ 目录下，逐脚本顶部含 docstring 与命令行用法）。

- src/_q_lib.py：数据读取（load_data）、距离矩阵、服务时间递推、统一 evaluate、$P_i(t_i)$ 早晚到二次罚、2-opt / Or-opt / Swap 修复算子——四问全部脚本共用此一份口径；
- src/01_q1_pure_python.py：Q1 Held–Karp 精确动态规划基线；
- src/02_q1_qubo_kaiwu_sdk.py：Q1 位置 one-hot QUBO 构造、Kaiwu SDK 模拟退火求解、adjust_qubo_matrix_precision 8-bit 调整、Ising 矩阵导出 CSV；

- `src/05_q2_pure_python.py` / `05c_q2_pure_python_v2.py`：Q2 多起点启发式 + LNS + 多 seed SA 求解 TSPTW；
- `src/09_q3_kaiwu_decompose.py`:Q3 滚动窗口分解 subQUBO（17/17/16）与 `solve_subqubo(Q, backend)` 抽象层；
- `src/14_q4_pure_python.py`：Q4 多起点 + 跨车 LNS + 单车 polish + K 扫描；
- `src/15b_q4_k8_blockdiag_cim.py`：Q4 $K = 8$ block-diag 7 段合并子 QUBO 配额优化方案；
- `src/13_all_q_cim_real.py`：CIM 真机统一调用入口（token 配置、矩阵上传、spin 解下载、哈密顿量曲线绘图、与 SDK SA 的对照）；
- `src/06_q1_sens_A_T0.py` / `14b_q4_K_sensitivity.py`：两类灵敏度扫描；
- `src/draw_*.py`：图表生成脚本，输出落到 `figures/` 与 `tables/`。

附录 D　真机日志与中间结果说明

本文累计向 CPQC-550 真机提交 **12 次任务**，所有提交矩阵的 ising 维度均 ≤ 550 比特，且经 `adjust_qubo_matrix_precision(Q, bit_width=8)` 8-bit 精度调整后上传。各任务的输入 / 返回与解码结果落盘于 `results/真机结果/`。任务清单见表 ??。

表 21: CIM CPQC-550 真机 12 次任务清单与产出文件

任务编号	对应问题	ising 维度	备注	产出文件路径（results/真机结果
R-Q1-005	Q1 (n=15)	226	单 task 10 sample	q1/cim_q1_*.zip / *.json / *.
R-Q2-006	Q2 (n=15)	226	单 task 10 sample	q2/cim_q2_*.zip / *.json / *.
R-Q3-seg1	Q3 段 1	290	配额异常未获返回	q3/seg1_failure.log（如实标注
R-Q3-seg2	Q3 段 2	290	单 task 10 sample	q3/cim_q3_seg2_*.zip / *.json
R-Q3-seg3	Q3 段 3	257	单 task 10 sample	q3/cim_q3_seg3_*.zip / *.json
R-Q4-V1∼V7	Q4 $K^* = 7$ 七车	≤ 65/车	共 7 次配额，每车 10 sample	q4/cim_q4_v{1..7}_*.zip 等
R-Q4-K8-bd	Q4 $K = 8$ block-diag	298	v2–v8 合并 1 次配额	q4/cim_q4_k8_blockdiag_*.zip

由表 ?? 可知：（i）所有真机任务的 ising 维度均不超过 550，可直接通过平台手动上传或 SDK `CIMOptimizer` 提交；（ii）Q3 段 1 因旧账号配额异常未获得有效返回，本文未补造数据，而以 `seg1_failure.log` 在结果目录中如实标注；（iii）每个真机 spin 解的长度 = ising 维度 = QUBO 维度 + 1（含 1 个辅助自旋），解码取 `spin[:nvar]`，最后 1 位辅助维丢弃，详见 `src/13_all_q_cim_real.py`；（iv）Q2 / Q3 / Q4 详细哈密顿量演化曲线 PDF 与原始 spin 解 zip 均位于 `results/真机结果/` 对应子目录下，可与正文图 ??/??/??/?? 原始数据交叉核验。

附录 E　问题二逐客户调度明细表

为压缩正文篇幅，问题二 15 客户的逐点服务开始时刻、时间窗 $[a_i, b_i]$ 与单点惩罚 P_i 详表（与正文表 ?? 对应）移至本附录；正文 §?? 仅保留汇总数值与甘特图。

表 22: 问题 2 单车辆调度（含时间窗违反）

客户	到达时刻	时间窗	早到	晚到	惩罚	离开时刻
2	2	[3,12]	1	0	10	4
13	5	[9,27]	4	0	160	7
6	8	[7,20]	0	0	0	10
5	11	[3,14]	0	0	0	13
8	15	[8,18]	0	0	0	17
7	19	[9,14]	0	5	500	21
11	23	[4,20]	0	3	180	25
10	26	[13,20]	0	6	720	28
1	30	[17,26]	0	4	320	32
9	34	[7,20]	0	14	3920	36
3	38	[14,18]	0	20	8000	40
12	41	[5,13]	0	28	15680	43
4	45	[9,25]	0	20	8000	47
15	51	[4,20]	0	31	19220	53
14	55	[4,18]	0	37	27380	57
					总运输时间	31
					时间窗惩罚总和	84090
					目标 J	84121